SPEAKSAFE – CYBERBULLYING AND HATE SPEECH DETECTION PLATFORM

A PROJECT REPORT

Submitted by

S. LISHANTH (310625149023)

AKASH V (310625149003)

in partial fulfillment for the award of the degree

of

BACHELOR OF ENGINEERING

in

COMPUTER SCIENCE AND ENGINEERING

(CYBER SECURITY)



EASWARI ENGINEERING COLLEGE

(AN AUTONOMOUS INSTITUTION) – AFFILIATED TO ANNA UNIVERSITY, CHENNAI

NOVEMBER 2026

EASWARI ENGINEERING COLLEGE

BONAFIDE CERTIFICATE

Certified that this project report titled **"SPEAKSAFE – CYBERBULLYING AND HATE SPEECH DETECTION PLATFORM"** is the bonafide work of **"S. LISHANTH (310625149023), AKASH V (310625149003)"** who carried out the project work under my supervision.

SIGNATURE	**SIGNATURE**
Dr. V. Balaji	[Guide Name]
HEAD OF THE DEPARTMENT	**SUPERVISOR**
Professor	[Designation]
Dept. of CSE (Cyber Security)	Dept. of CSE (Cyber Security)
Easwari Engineering College	Easwari Engineering College
Ramapuram, Chennai.	Ramapuram, Chennai.

EASWARI ENGINEERING COLLEGE

Certificate of Viva-voce Examination

This is to certify that the Viva Voce Examination of the Project Work entitled "_____" submitted by Mr./Ms. _____ (Reg. No. _____) in partial fulfillment of the requirements for the award of the degree of _____ was conducted on _____ (Date) in the Department of _____, Easwari Engineering College, Ramapuram – Chennai.

Internal Examiner	**External Examiner**
Name :	Name :
Designation :	Designation :
Address :	Address :

Date :

ABSTRACT

Social media, messaging applications and online forums have become the main places where students and young people talk to each other. Along with their benefits, these platforms have made cyberbullying, harassment and hate speech a widespread problem. Abusive messages spread within minutes, follow victims home and can seriously affect mental health. Manual moderation cannot keep up with the volume of messages, and simple keyword filters are easy to bypass and often block harmless text.

This project presents **SpeakSafe**, a cyberbullying and hate speech detection platform that reads text in the web browser and detects **insults, threats and hate speech** in real time. Each message is cleaned and normalised using Natural Language Processing (lower-casing, link and mention removal, contraction expansion, leetspeak and masked swear-word normalisation, tokenisation with Apache OpenNLP and Porter stemming), converted into TF-IDF features over single words and word pairs, and classified by a **Linear Support Vector Machine** trained in Java. The system produces a **toxicity score from 0 to 100** and a category label (safe, insult, threat or hate speech), and decides an action: show the message, warn the user, or blur the message and flag it for moderators.

The core engine is implemented in **Java with Spring Boot** and exposed as a REST API, with flagged messages stored in an **SQLite** database and reviewed on a moderator dashboard. A **Chrome browser extension** (HTML, CSS and JavaScript) warns users before they send a hurtful message and blurs toxic messages on any web page with options to reveal or report them. The trained model can also be exported to the extension so that it works without a server; this browser version was verified to give identical results to the Java engine on 2,823 test messages.

On a labelled dataset of 1520 messages, the Linear SVM achieved an accuracy of 95.7% (macro F1-score 93.4%) and Naive Bayes 96.4% (macro F1-score 94.2%) on a 20% test split, with an average processing time below 0.1 ms per message. On 125 hand-written messages and article paragraphs that the model never saw during training, the system correctly left 76 of 78 harmless texts untouched and caught 46 of 47 toxic ones, including disguised swear words such as “f*ck” and “sh*t”. SpeakSafe shows that a lightweight, beginner-friendly Java system can provide practical, real-time protection against online abuse on an ordinary personal computer, without GPUs or very large datasets.

ACKNOWLEDGEMENT

We express our sincere gratitude to the Principal **Dr. P. Deiva Sundari** and the Management of our college for providing us with the opportunity to carry out this project work.

We convey our heartfelt thanks to the **Head of the Department Dr. V. Balaji**, Department of Computer Science and Engineering (Cyber Security), for the continuous encouragement and support.

We are extremely thankful to our **Project Guide [Guide Name]**, Department of Computer Science and Engineering (Cyber Security), for valuable guidance, suggestions and motivation throughout the completion of this project.

We would also like to extend our gratitude to all the faculty members of the Department of Computer Science and Engineering (Cyber Security) for their support.

Finally, we thank our parents and friends for their constant encouragement and help during the course of this project.

TABLE OF CONTENTS

CHAPTER NO.	TITLE	PAGE NO.
	ABSTRACT	**iv**
	ACKNOWLEDGEMENT	**v**
	LIST OF TABLES	**vii**
	LIST OF FIGURES	**viii**
	LIST OF ABBREVIATIONS	**ix**
1.	**INTRODUCTION**	**1**
	1.1 OVERVIEW	1
	1.2 PROBLEM DEFINITION	2
	1.3 OBJECTIVE	3
	1.4 SCOPE OF THE PROJECT	4
	1.5 MOTIVATION	5
2.	**RELATED WORK**	**6**
	2.1 LITERATURE REVIEW	6
	2.2 EXISTING SYSTEM	7
	2.3 PROPOSED SYSTEM	9
	2.4 ADVANTAGES OF PROPOSED SYSTEM	10
3.	**SYSTEM ANALYSIS**	**11**
	3.1 SYSTEM REQUIREMENTS	11
	3.2 SOFTWARE AND HARDWARE REQUIREMENTS	12
	3.3 SYSTEM REQUIREMENT SPECIFICATION (SRS)	13
4.	**SYSTEM DESIGN**	**15**
	4.1 SYSTEM ARCHITECTURE	15
	4.2 MODULE DESCRIPTION	17
	4.3 USE CASE DIAGRAM	18
	4.4 CLASS DIAGRAM	19
	4.5 SEQUENCE DIAGRAM	20
	4.6 DATABASE DESIGN	21
5.	**IMPLEMENTATION**	**22**
	5.1 IMPLEMENTATION OVERVIEW	22
	5.2 DATASET	22
	5.3 KEY FUNCTIONS (CODE EXPLANATION)	23
	5.4 RESULTS AND DISCUSSION	26
6.	**CONCLUSION & FUTURE WORK**	**33**
	6.1 CONCLUSION	33
	6.2 FUTURE ENHANCEMENTS	34
	APPENDIX 1: SOURCE CODE	**35**
	APPENDIX 2: SAMPLE OUTPUT	**71**
	REFERENCES	**72**

LIST OF TABLES

TABLE NO.	TITLE	PAGE NO.
1.1	Sample Messages Analysed by SpeakSafe	3
2.1	Advantages of the Proposed System	10
3.1	Functional Requirements	11
3.2	Non-Functional Requirements	12
3.3	Software Requirements	12
3.4	Hardware Requirements	13
4.1	Module Description	17
4.2	Structure of the flagged_messages Table	21
5.1	Composition of the Dataset	23
5.2	REST API Endpoints	25
5.3	Comparison of Naive Bayes and Linear SVM (20% test split)	26
5.4	Per-Category Results of the Linear SVM	26
5.5	Confusion Matrix of the Linear SVM (rows = actual, columns = predicted)	27
5.6	Results on Unseen Text (Linear SVM)	27

LIST OF FIGURES

FIGURE NO.	TITLE	PAGE NO.
1.1	SpeakSafe extension blurring toxic messages in a group chat and warning while typing	2
4.1	System Architecture	16
4.2	System Flow Diagram	18
4.3	Use Case Diagram	19
4.4	Class Diagram	20
4.5	Sequence Diagram	21
5.1	Web Message Checker Showing Score, Category and Probabilities	29
5.2	Extension Warning the User While Typing in a Chat	29
5.3	Blurred Message Revealed with the Show Button and Reported	30
5.4	Extension Popup Running the Built-in Model	30
5.5	Moderator Dashboard with Flagged Messages and Model Evaluation	31
5.6	Flagged Messages Page of the Extension (No Server Mode)	32

LIST OF ABBREVIATIONS

- **API** – Application Programming Interface
- **BERT** – Bidirectional Encoder Representations from Transformers
- **CNN** – Convolutional Neural Network
- **CSV** – Comma Separated Values
- **DOM** – Document Object Model
- **GPU** – Graphics Processing Unit
- **HTML** – HyperText Markup Language
- **HTTP** – HyperText Transfer Protocol
- **IDF** – Inverse Document Frequency
- **JDK** – Java Development Kit
- **JSON** – JavaScript Object Notation
- **LSTM** – Long Short-Term Memory
- **ML** – Machine Learning
- **NB** – Naive Bayes
- **NLP** – Natural Language Processing
- **REST** – Representational State Transfer
- **SQL** – Structured Query Language
- **SRS** – Software Requirement Specification
- **SVM** – Support Vector Machine
- **TF-IDF** – Term Frequency – Inverse Document Frequency
- **UI** – User Interface
- **URL** – Uniform Resource Locator

CHAPTER 1
INTRODUCTION

1.1 OVERVIEW

In today's connected world, a large part of everyday communication happens online. Students use WhatsApp groups, Instagram comments, YouTube discussions, online games and college forums to share notes, plan events and stay in touch. Along with these benefits, online platforms have also made it easy to insult, threaten and humiliate others. **Cyberbullying** is the use of digital technology to repeatedly harass, threaten or embarrass another person, and **hate speech** is abusive language that attacks people because of their religion, ethnicity, nationality, gender, sexuality or disability.

Unlike playground bullying, online abuse can reach a victim at any time of the day, can be shared hundreds of times within minutes and can stay visible for a long time. Victims often feel anxious, isolated and ashamed, and many never report what happened. Platform moderators cannot read every message, so harmful content often remains online until someone complains.

Natural Language Processing (NLP) and **Machine Learning (ML)** allow a computer to read text and recognise patterns of abusive, offensive or threatening language automatically. SpeakSafe uses these techniques to build a **Cyberbullying and Hate Speech Detection Platform in Java** that is simple enough for a beginner to build, understand and extend. The detection engine is packaged together with a **browser extension**, so that it protects users directly while they browse or chat online.

Key Features:

1. **Real-time warning while typing:**
    - When the user types a message in any text box, SpeakSafe checks it and shows a warning such as "this message looks like an insult (toxicity 94/100)" before the message is sent.
2. **Automatic blurring of toxic messages:**
    - Comments and chat messages on any web page are scanned. Toxic ones are blurred and labelled, with **Show** and **Report** buttons.
3. **Toxicity score and category:**
    - Every message receives a graded score from 0 (safe) to 100 (very toxic) and a category: Safe, Insult, Threat or Hate speech.
4. **Reporting and moderation:**
    - Highly toxic messages can be flagged automatically, and users can report messages. Moderators review them on a dashboard with statistics per category.

5. **Lightweight and offline-capable:**
    - The model runs on an ordinary personal computer without a GPU. The trained model can also run inside the browser extension without any server.

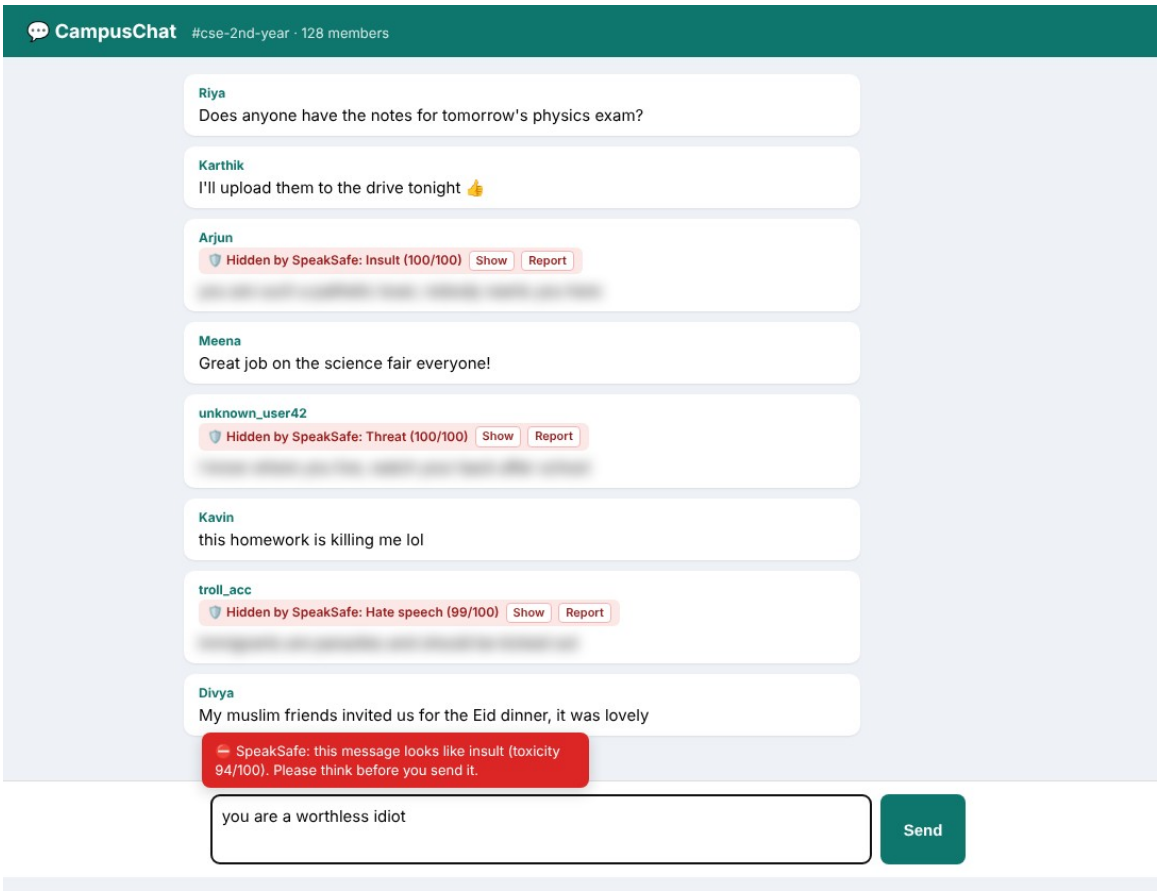


Fig 1.1 – SpeakSafe extension blurring toxic messages in a group chat and warning while typing

1.2 PROBLEM DEFINITION

Detecting and handling abusive messages manually has several drawbacks:

1. **Volume:**
    - Millions of posts, comments and messages are created every day. Human moderators can only review a small fraction of them, usually after the harm is done.
2. **Delay:**
    - Abuse is usually removed only after someone reports it. By then the victim and many others have already seen it.
3. **Weak keyword filters:**

- Simple lists of bad words are easily bypassed with spellings such as “l0ser” or “stuuupid”, and they wrongly block harmless sentences such as “this homework is killing me”.

4. **No graded severity:**
   - Most simple filters only say “blocked” or “allowed”. They do not tell how severe a message is or whether it is an insult, a threat or hate speech.
5. **Heavy research models:**
   - State-of-the-art deep learning models are accurate but need large datasets, GPUs and complex deployment, which makes them hard for students and small organisations to use.

Problem Statement:

There is a need for a lightweight, real-time and easy-to-understand system that reads user-generated text, assigns it a toxicity score and a category (insult, threat, hate speech or safe), and automatically warns, blurs or reports harmful messages directly inside the user’s web browser.

Table 1.1 shows how SpeakSafe classifies a few sample messages. None of these sentences were part of the training data.

Message	Score	Category	Action
thanks for helping me with the homework	2	SAFE	NONE
this homework is killing me lol	7	SAFE	NONE
u r such a l0ser	99	INSULT	BLUR
f*ck off	100	INSULT	BLUR
I know where you live, watch your back after school	100	THREAT	BLUR
immigrants are parasites and should be kicked out	99	HATE SPEECH	BLUR
My muslim friends invited us for the Eid dinner, it was lovely	7	SAFE	NONE

Table 1.1 – Sample Messages Analysed by SpeakSafe

1.3 OBJECTIVE

The main objective of SpeakSafe is to build a simple, reliable and automated system that detects cyberbullying and hate speech in text and helps users and moderators act on it. The objectives are:

1. **Detect abusive language using NLP:**

- Identify cyberbullying, hate speech and offensive language in user-generated text such as chats, comments and posts.
- Handle common tricks like leetspeak (“1d10t”), repeated letters (“stuuupid”), masked swear words (“f*ck”) and contractions (“you’re”).

2. **Score and categorise every message:**
   - Assign each message a toxicity score from 0 to 100.
   - Classify harmful messages into insult, threat or hate speech.
3. **Act automatically:**
   - Warn the user while typing, blur toxic messages on the page and flag or report them for moderators.
4. **Use beginner-friendly Java tools:**
   - Build the core engine in Java with Spring Boot, Apache OpenNLP and Maven, with well-commented, modular code.
5. **Package the system as a browser extension:**
   - Provide a Chrome extension that works on any website or chat platform.
6. **Evaluate the models:**
   - Measure accuracy, precision, recall and F1-score, and compare a classical Naive Bayes model with a Linear SVM.
7. **Promote digital well-being:**
   - Encourage safer and kinder online interactions for users of all ages.

1.4 SCOPE OF THE PROJECT

In Scope:

- Detection of insults, threats and hate speech in English text.
- Toxicity score (0–100), category label and action (none, warn, blur) for every message.
- Java Spring Boot REST API, SQLite storage of flagged messages and a moderator dashboard.
- Chrome extension that warns while typing, blurs messages on web pages and reports them.
- Built-in browser model so that the extension also works without the server.
- Comparison of Naive Bayes and Linear SVM classifiers with standard evaluation metrics.

Out of Scope:

- Languages other than English and code-mixed text (for example Tanglish or Hinglish).

- Images, videos, memes and voice messages.
- Mobile applications (Android or iOS keyboards).
- Automatic removal of content from third-party platforms; SpeakSafe only acts in the user's own browser.
- Large transformer models such as BERT that need GPUs.

1.5 MOTIVATION

The motivation for this project includes:

1. **Student safety:**
    - Many students experience or witness online bullying in class groups and social media. A tool that warns and protects them in real time can reduce harm.
2. **Prevention instead of punishment:**
    - Warning a person before they send a hurtful message gives them a chance to reconsider, which is better than removing the message afterwards.
3. **Educational value:**
    - The project applies Java, object-oriented programming, NLP, machine learning, REST APIs, databases and browser extensions in one real-world system.
4. **Cyber security relevance:**
    - Harassment, threats and hate speech are important parts of online safety and cyber security, which is the focus of our department.
5. **Future potential:**
    - The system can later be extended with larger datasets, more languages and deep learning models.

CHAPTER 2
RELATED WORK

2.1 LITERATURE REVIEW

Automatic detection of cyberbullying and hate speech has been studied widely in NLP and machine learning research. This review covers the main approaches, from classical machine learning to deep learning and transformer models, and identifies the gap that this project addresses.

1. Classical Machine Learning for Cyberbullying Detection

Alabdulwahab, Haq and Alshehri (2023) studied cyberbullying detection using machine learning and deep learning. They extracted NLP features from text and compared K-Nearest Neighbours, Support Vector Machine, Naive Bayes, Decision Tree and Random Forest classifiers with a Convolutional Neural Network (CNN). Classical machine learning gave a fast and simple baseline, while the CNN improved accuracy but needed more data and tuning. This supports our choice of starting with classical models that are fast and easy to deploy.

Dinakar, Reichart and Lieberman (2011) were among the first to model textual cyberbullying. They observed that bullying often targets sensitive topics such as appearance, intelligence, race and sexuality, and that topic-sensitive classifiers work better than one general classifier. This idea of separate categories inspired our use of separate labels for insults, threats and hate speech.

2. Hate Speech and Offensive Language

Davidson, Warmsley, Macy and Weber (2017) built a dataset of tweets labelled as hate speech, offensive language or neither, and trained classifiers on word n-gram features. Their key finding was that separating hate speech from general offensive language is difficult, and that many messages are wrongly classified because they contain offensive words used in a non-hateful way.

Waseem and Hovy (2016) studied predictive features for hate speech on Twitter and found that character n-grams were strong features. **Schmidt and Wiegand (2017)** surveyed hate speech detection and summarised the common features used: bag-of-words, n-grams, lexical resources, sentiment and knowledge-based features, together with classifiers such as SVM and logistic regression.

3. Deep Learning-Based Detection

The review published in Future Internet (2023) surveyed deep-learning-based cyberbullying detection using CNN, RNN, LSTM and hybrid architectures. It concluded that deep learning

generally outperforms classical machine learning, but dataset quality, class imbalance and real-time deployment remain open issues.

Nath, Karim and Miraz (2024) applied deep learning to cyberbullying detection in the Bangla language, showing that the problem extends beyond English and that language-specific data is needed.

4. Transformer Models

Khafajeh (2024) compared CNN, LSTM and BERT models for cyberbullying detection in social networks on a public benchmark dataset and found that BERT-based models gave the best accuracy. **Devlin et al. (2019)** introduced BERT, a transformer pre-trained on large text corpora. Such models understand context very well, but they are large, need GPUs for training and are harder for beginners to deploy in a browser.

5. Large-Scale Toxicity Detection

Wulczyn, Thain and Dixon (2017) analysed personal attacks on Wikipedia talk pages at large scale using crowd-sourced labels and machine learning. This work led to the Jigsaw Toxic Comment Classification dataset, whose labels (toxic, insult, threat, identity hate) are similar to the categories used in this project.

6. Surveys of Automated Cyberbullying Detection

Salawu, He and Lumsden (2020) surveyed approaches to automated cyberbullying detection and grouped them into machine learning, rule-based, mixed-initiative and deep learning approaches. They highlighted the lack of tools that are deployed in real-world settings where users can actually benefit from them.

Conclusion of the survey:

The reviewed studies show that classical machine learning with n-gram features gives a fast, lightweight baseline, while deep learning and transformers give the highest accuracy at the cost of data, hardware and complexity. Most existing work evaluates models offline on fixed datasets, and very few works provide a simple, real-time, deployable tool such as a browser extension. **SpeakSafe addresses this gap with a lightweight, Java-based, beginner-friendly detection engine that works live inside the browser.**

2.2 EXISTING SYSTEM

Existing approaches for dealing with cyberbullying and hate speech fall into the following categories:

1. **Manual moderation:**
    - Moderators read reported messages and remove harmful ones.

- **Advantages:** Human judgement understands context and sarcasm.
- **Limitations:** Slow, expensive, cannot scale to millions of messages, and moderators themselves are exposed to harmful content.
- **Example:** A class WhatsApp group admin removes an abusive message only after several students complain.

2. **Keyword (blocklist) filters:**
   - A list of bad words is matched against each message.
   - **Advantages:** Very simple and fast.
   - **Limitations:** Easily bypassed with misspellings and leetspeak; blocks harmless sentences; cannot detect threats or hate speech written without bad words; no severity score.
   - **Example:** “you are a l0ser” passes the filter while “this homework is killing me” is blocked.
3. **Built-in platform moderation:**
   - Large platforms use internal systems to hide offensive comments.
   - **Advantages:** Automatic and integrated.
   - **Limitations:** Only work inside that one platform, are not transparent and cannot be studied or customised by users.
4. **Cloud toxicity APIs:**
   - Online services score comments for toxicity using large models.
   - **Advantages:** Accurate and maintained by large teams.
   - **Limitations:** Require internet access and API keys, send private messages to a third party, and may have usage limits.
5. **Research models (deep learning / BERT):**
   - Published models achieve high accuracy on benchmark datasets.
   - **Limitations:** Need GPUs and large datasets, and are rarely packaged as tools that ordinary users can install.

Observation:

- Manual and keyword-based methods are simple but ineffective at scale.
- Platform and cloud solutions are closed, online-only or raise privacy concerns.
- A lightweight, transparent, real-time tool that runs on the user’s own computer is needed.

2.3 PROPOSED SYSTEM

The proposed system, **SpeakSafe**, is a Java-based cyberbullying and hate speech detection platform packaged with a Chrome browser extension.

Main Features:

1. **NLP preprocessing:**
    - Cleans text, removes links and @mentions, expands contractions, undoes leetspeak and masked swear words, collapses repeated letters, tokenises with Apache OpenNLP, removes stopwords and stems words with the Porter stemmer.
2. **TF-IDF feature extraction:**
    - Represents each message using TF-IDF weights of single words and pairs of neighbouring words.
3. **Machine learning classification:**
    - A Linear SVM (one-vs-rest) predicts the probability of each category. A Naive Bayes model is also implemented for comparison.
4. **Toxicity score and action:**
    - Score = $(1 - P(\text{safe})) \times 100$. Scores of 50 or more trigger a warning; scores of 70 or more blur the message and flag it.
5. **Browser extension:**
    - Warns while typing, blurs toxic messages, offers Show and Report buttons, and works without a server using the exported model.
6. **Moderation and storage:**
    - Flagged and reported messages are stored in SQLite and shown on a moderator dashboard.

2.4 ADVANTAGES OF PROPOSED SYSTEM

Feature	Advantage
Real-time	Warns before a message is sent and blurs abuse as soon as it appears
Graded score	Score from 0 to 100 shows how severe a message is, not just yes/no
Categories	Separates insults, threats and hate speech for better moderation
Robust to tricks	Handles leetspeak, masked swear words, repeated letters and contractions
Lightweight	Runs on a normal PC; no GPU; under 0.1 ms per message
Privacy	Text is processed locally on the user’s computer, not sent to a third party
Offline	Built-in model works in the browser without a server or internet
Educational	Modular, commented Java code that beginners can understand and extend

Table 2.1 – Advantages of the Proposed System

CHAPTER 3
SYSTEM ANALYSIS

This chapter analyses the requirements of SpeakSafe, both functional and non-functional, defines the software and hardware needed to develop and run it, and gives the system requirement specification.

3.1 SYSTEM REQUIREMENTS

The proposed system is designed to be lightweight, real-time and beginner-friendly. The following requirements ensure proper functioning and usability.

FUNCTIONAL REQUIREMENTS

Functional requirements describe what the system should do.

ID	Requirement Description
FR1	The system must accept a text message and return a toxicity score (0–100), a category and an action.
FR2	The system must preprocess text: lower-case, remove links and mentions, expand contractions, normalise leetspeak and masked swear words, tokenise, remove stopwords and stem.
FR3	The system must classify messages as Safe, Insult, Threat or Hate speech.
FR4	The extension must warn the user while typing when a message scores 50 or more.
FR5	The extension must blur messages on a web page that score 70 or more and offer Show and Report options.
FR6	The system must store flagged and reported messages with score, category, page and time.
FR7	The moderator dashboard must list flagged messages, show statistics per category and allow dismissing a report.
FR8	The system must evaluate Naive Bayes and Linear SVM models and report accuracy, precision, recall and F1-score.
FR9	The extension must keep working with a built-in model when the Java server is not running.

Table 3.1 – Functional Requirements

NON-FUNCTIONAL REQUIREMENTS

Non-functional requirements describe the quality attributes of the system.

ID	Attribute	Requirement Description
NFR1	Performance	Each message must be analysed in well under 100 ms so that warnings appear while typing.
NFR2	Usability	Warnings and labels must be clear; the extension must be installable in a few minutes.
NFR3	Reliability	Invalid input (empty, very long, emoji-only text) must be handled without errors.
NFR4	Privacy	Messages must be processed locally; no data is sent to external services.
NFR5	Portability	The backend must run on Windows, Linux and macOS with Java 17; the extension on Chromium browsers.
NFR6	Security	Stored messages must be displayed as plain text so that they cannot inject scripts into the dashboard.
NFR7	Maintainability	Code must be modular and commented so that models and datasets can be replaced easily.

Table 3.2 – Non-Functional Requirements

3.2 SOFTWARE AND HARDWARE REQUIREMENTS

SpeakSafe is lightweight and works on ordinary computers.

SOFTWARE REQUIREMENTS

Software Component	Requirement
Operating System	Windows 10/11, Linux or macOS
Programming Language	Java 17 or above (JDK); JavaScript for the extension
Backend Framework	Spring Boot 3.3
NLP Library	Apache OpenNLP 2.5 (tokenizer, Porter stemmer)
Database	SQLite 3 (sqlite-jdbc driver)
Build Tool	Apache Maven (Maven wrapper included)
Browser	Google Chrome, Microsoft Edge or Brave (Manifest V3 extensions)
Code Editor / IDE	VS Code, IntelliJ IDEA or Eclipse
Testing	JUnit 5
Version Control	Git and GitHub

Table 3.3 – Software Requirements

HARDWARE REQUIREMENTS

Hardware Component	Requirement
Processor	Intel Core i3 or equivalent
RAM	4 GB or above
Storage	500 MB free disk space
GPU	Not required
Network	Not required for detection (only for downloading libraries once)
Input / Display	Keyboard, mouse and a standard monitor

Table 3.4 – Hardware Requirements

3.3 SYSTEM REQUIREMENT SPECIFICATION (SRS)

3.3.1 Users

- **End users** – students and young people who chat, comment and browse online.
- **Moderators** – teachers, group administrators or platform moderators who review flagged messages.
- **Developers / researchers** – who retrain or evaluate the models.

3.3.2 System Scope

The system reads English text from web pages and text boxes in the browser, scores it for toxicity, categorises it and acts on it. It stores flagged messages for review and provides tools to evaluate the models.

3.3.3 Inputs

- Text typed by the user in any text box or editable area.
- Text of comments and messages displayed on web pages.
- Text entered in the web message checker.
- Labelled training dataset (CSV file with label and text).
- User settings (protection on/off, warn while typing, blur on pages, auto-flag).

3.3.4 Outputs

- Toxicity score (0–100) and probability of each category.
- Category label: Safe, Insult, Threat or Hate speech.
- Action: none, warning bubble, or blurred message with Show and Report buttons.
- Flagged message records and statistics on the moderator dashboard.
- Evaluation report with accuracy, precision, recall, F1-score and confusion matrix.

3.3.5 Constraints

- Only English text is supported.
- The quality of detection depends on the size and quality of the training dataset.
- Browser extensions cannot read text inside images or some protected pages (for example the Chrome Web Store).

CHAPTER 4
SYSTEM DESIGN

4.1 SYSTEM ARCHITECTURE

The architecture of SpeakSafe focuses on building a simple, modular and real-time system for detecting abusive text. Each major function is placed in its own module, which improves readability, maintainability and scalability. The system is organised into three layers: the client layer, the application layer and the data layer (Fig 4.1).

The **client layer** is the Chrome extension. The content script (content.js) runs inside every web page. It captures text typed by the user and the text of messages displayed on the page, and it performs the visible actions: showing a warning bubble, blurring a message and adding Show and Report buttons. The background service worker (background.js) receives all requests from the content script and decides where to send them: to the Java server if it is running, or to the built-in model otherwise. The popup and reports pages provide settings, counters, a quick test box and the list of flagged messages.

The **application layer** is the Java Spring Boot backend. The DetectionController exposes a REST API. The ToxicityDetector converts model probabilities into a score, category and action. The DetectionPipeline connects the three processing modules in order: TextPreprocessor, TfIdfVectorizer and the Classifier (Linear SVM or Naive Bayes). The ReportRepository stores flagged messages, and the web user interface provides the message checker and moderator dashboard.

The **data layer** contains the labelled training dataset (CSV) used to train the model when the server starts, and the SQLite database that stores flagged and reported messages.

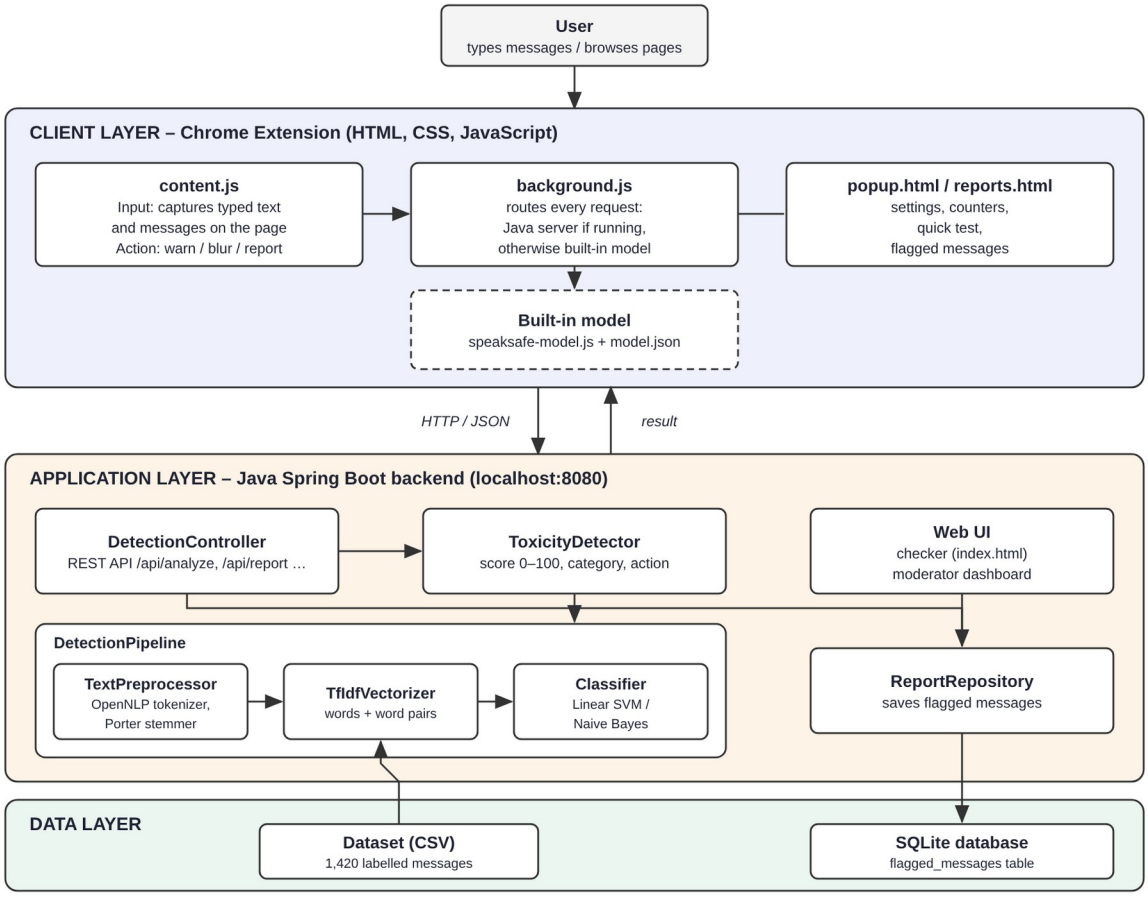


Fig 4.1 – System Architecture

The architecture promotes the following design goals:

1. **Simplicity:**
   - Each class has one clear responsibility and is short and well commented.
2. **Real-time response:**
   - Text is analysed in a fraction of a millisecond, so warnings appear while the user is still typing.
3. **Privacy:**
   - All processing happens on the user's own computer.
4. **Replaceability:**
   - The Classifier interface allows Naive Bayes, SVM or a future deep learning model to be swapped without changing the rest of the system.
5. **Graceful degradation:**
   - If the server is unavailable, the extension continues working with the built-in model.

4.2 MODULE DESCRIPTION

The modules of SpeakSafe follow the modules proposed in the project review: Input, Preprocessing, Feature Extraction, Classification, Categorization and Action, supported by storage and evaluation modules.

Module	Classes / Files	Role in System
Input Module	content.js, background.js, DetectionController.java	Captures typed text and page messages, and receives them through the REST API
Preprocessing Module	TextPreprocessor.java	Cleans and normalises text and converts it into word stems using Apache OpenNLP
Feature Extraction Module	TfIdfVectorizer.java	Converts words and word pairs into a normalised TF-IDF vector
Classification Module	Classifier.java, SvmClassifier.java, NaiveBayesClassifier.java	Predicts the probability of each category
Categorization Module	ToxicityDetector.java	Computes the 0–100 score and chooses the category
Action Module	ToxicityDetector.java, content.js	Decides NONE / WARN / BLUR and shows warnings, blurs and report buttons
Database Module	ReportRepository.java, schema.sql	Stores flagged and reported messages in SQLite
Training and Evaluation	DatasetLoader.java, DetectionPipeline.java, ModelEvaluator.java, ModelExporter.java	Loads data, trains models, measures metrics and exports the model for the browser
Built-in Browser Model	speaksafe-model.js, model.json	Runs the same trained model inside the extension without a server

Table 4.1 – Module Description

SYSTEM FLOW

- The user types a message or opens a page; content.js captures the text.
- background.js sends the text to the Java REST API, or to the built-in model if the server is not running.
- The text is preprocessed, converted to TF-IDF features and classified by the Linear SVM.
- The toxicity score is calculated. Below 50 the message is safe; from 50 the most likely toxic category is chosen.

- Scores from 50 to 69 produce a warning; scores of 70 or more blur the message and flag it for moderators.
- content.js shows the result to the user.

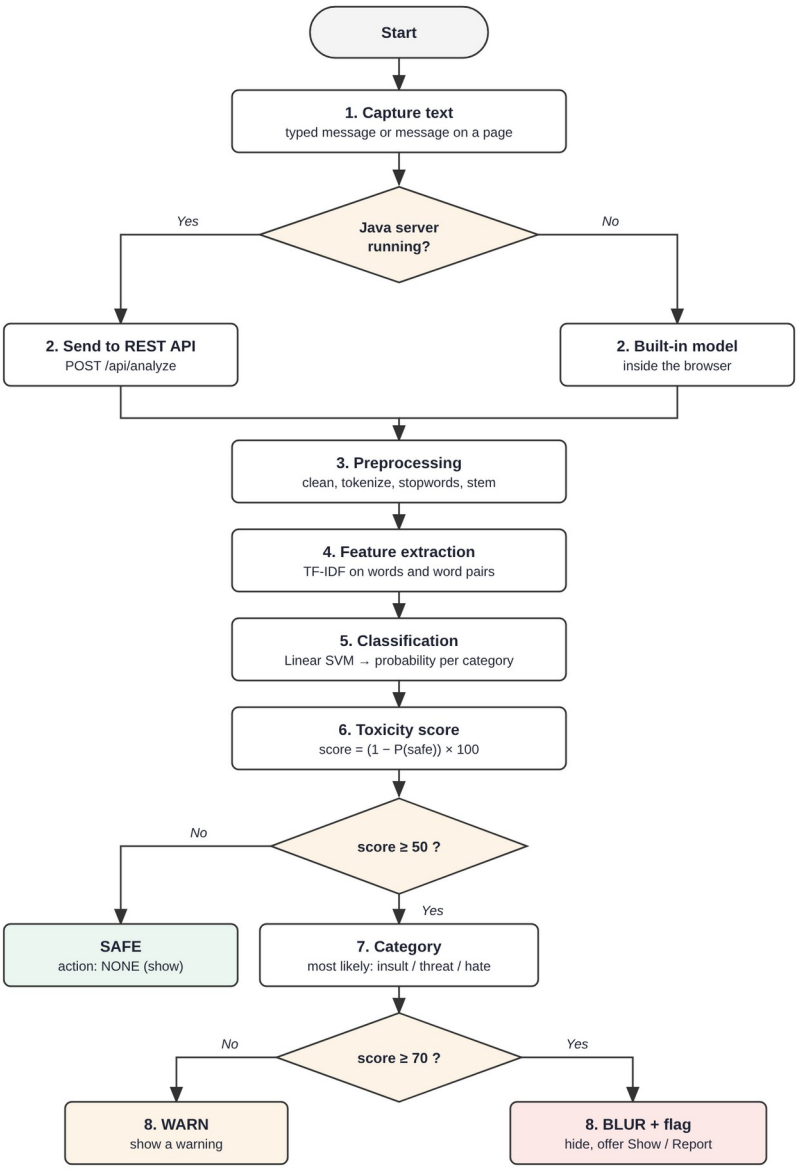


Fig 4.2 – System Flow Diagram

4.3 USE CASE DIAGRAM

The use case diagram (Fig 4.3) shows how users interact with SpeakSafe. There are two actors. The **User** types messages and receives warnings, browses pages where toxic text is blurred, reveals or reports messages, checks messages with the web form and changes extension settings. The **Moderator** views flagged messages and statistics, dismisses reports and runs the model evaluation. Several user actions include the common use case “Analyze & score text”.

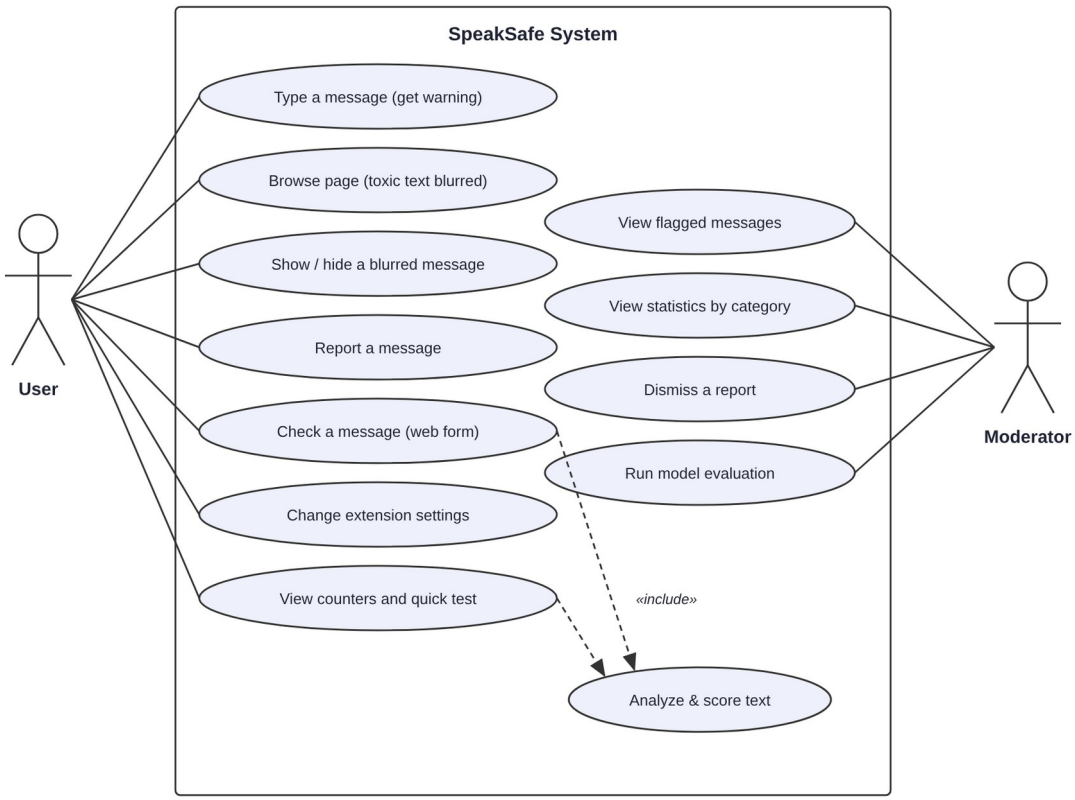


Fig 4.3 – Use Case Diagram

4.4 CLASS DIAGRAM

The class diagram (Fig 4.4) shows the main Java classes and their relationships. DetectionController uses ToxicityDetector and ReportRepository. ToxicityDetector uses a DetectionPipeline, which contains a TextPreprocessor, a TfIdfVectorizer and a Classifier. NaiveBayesClassifier and SvmClassifier both implement the Classifier interface. AnalysisResult and Report are Java records that carry data, while Category and Action are enumerations.

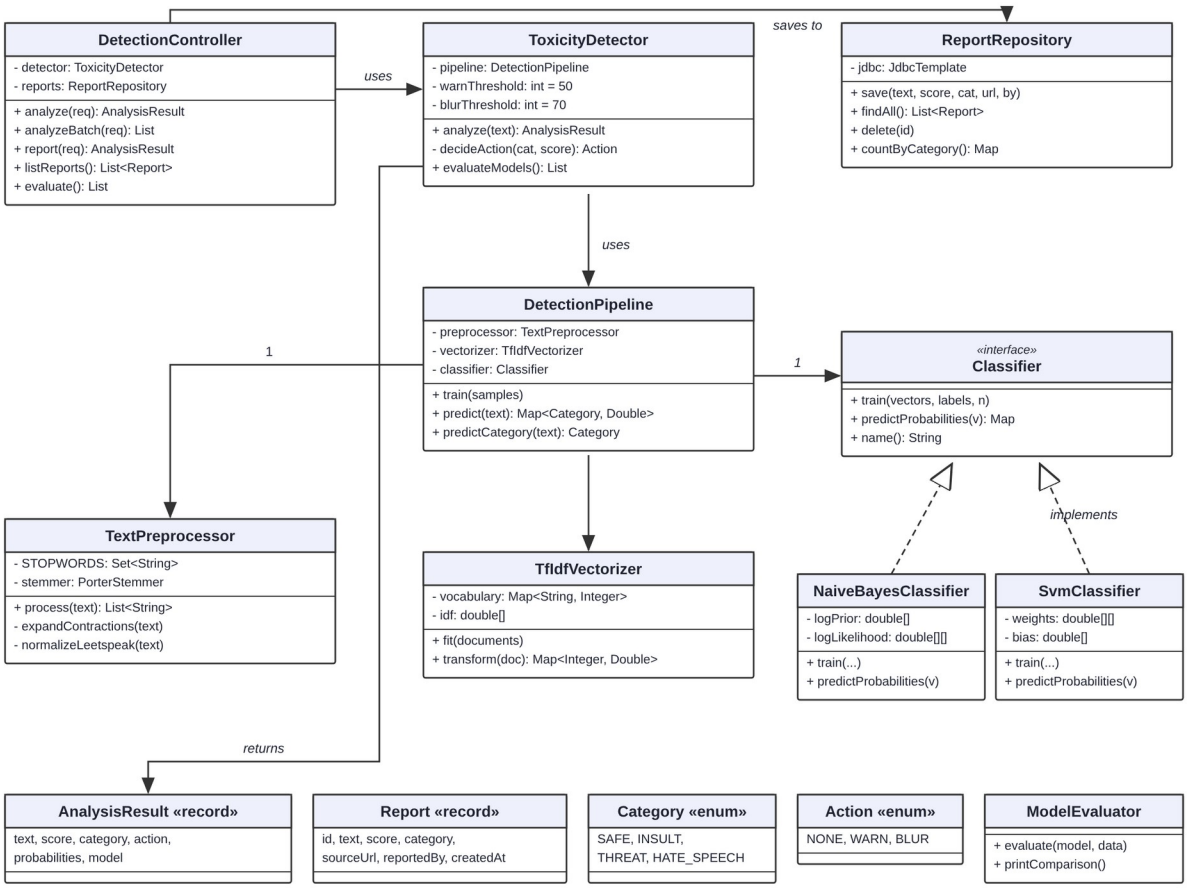


Fig 4.4 – Class Diagram

4.5 SEQUENCE DIAGRAM

The sequence diagram (Fig 4.5) shows the order of interactions when a message is analysed. The content script sends the text to the background worker, which calls the REST API. The controller asks the ToxicityDetector to analyse the text; the detector gets the probabilities from the DetectionPipeline and calculates the score, category and action. If the message is blurred and auto-flag is on, it is saved by the ReportRepository. The result travels back to the content script, which shows a warning or blurs the message. When the server is offline, the background worker uses the built-in model instead.

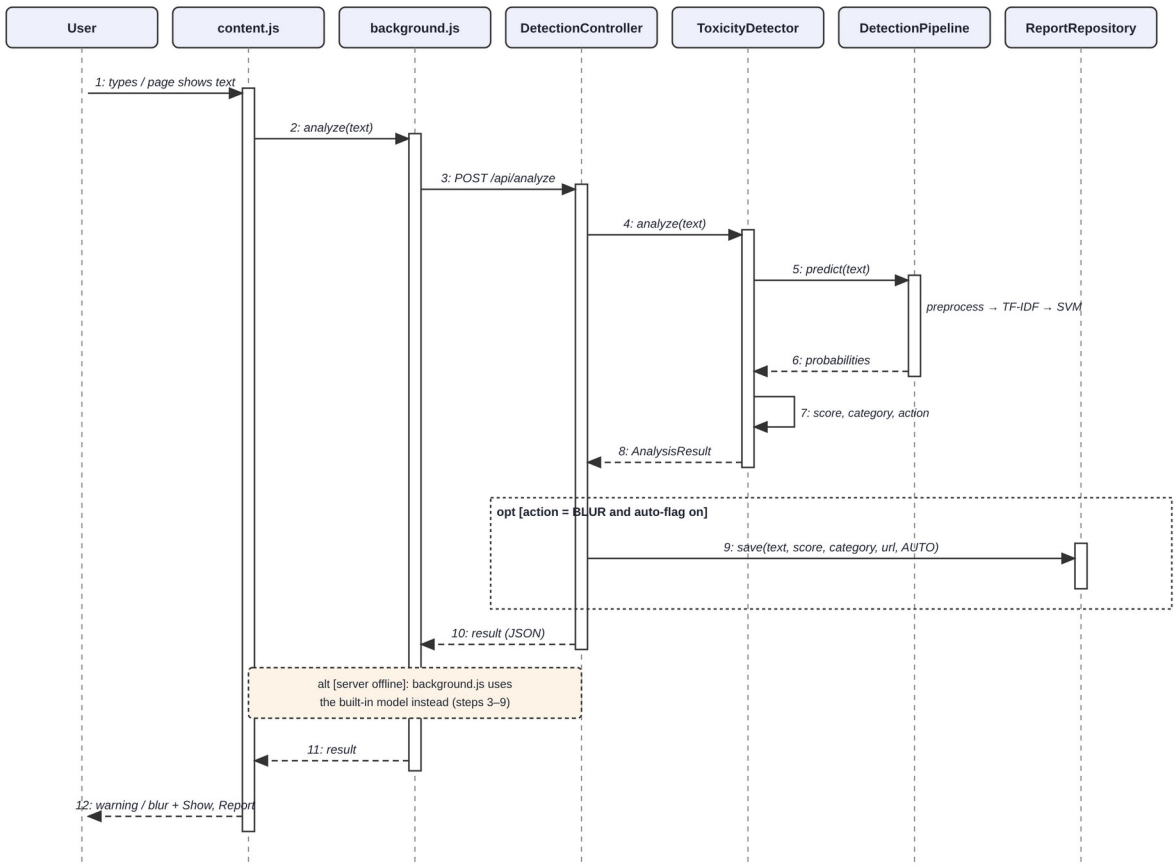


Fig 4.5 – Sequence Diagram

4.6 DATABASE DESIGN

Flagged and reported messages are stored in a single SQLite table called flagged_messages. A unique index on (text, source_url, reported_by) prevents the same message from the same page being stored twice.

Column	Type	Description
id	INTEGER (PK)	Auto-increment primary key
text	TEXT	The flagged message
score	INTEGER	Toxicity score from 0 to 100
category	TEXT	INSULT, THREAT or HATE_SPEECH
source_url	TEXT	Web page where the message was found
reported_by	TEXT	AUTO (flagged by the system) or USER (reported by a person)
created_at	TEXT	Date and time of flagging

Table 4.2 – Structure of the flagged_messages Table

CHAPTER 5
IMPLEMENTATION

5.1 IMPLEMENTATION OVERVIEW

SpeakSafe has been implemented as a Java Spring Boot application together with a Chrome extension. The backend is built with Maven and contains about twenty short Java classes organised into packages: model, nlp, ml, service and controller. The extension is written in plain HTML, CSS and JavaScript using Manifest V3.

The project follows a modular approach:

1. **TextPreprocessor.java** – cleans and normalises text and produces word stems.
2. **TfIdfVectorizer.java** – learns the vocabulary and IDF weights and converts text into TF-IDF vectors.
3. **SvmClassifier.java / NaiveBayesClassifier.java** – the two classification models.
4. **DetectionPipeline.java** – connects preprocessing, feature extraction and classification.
5. **ToxicityDetector.java** – trains the chosen model at start-up and produces the score, category and action.
6. **DetectionController.java** – REST API used by the extension and the web pages.
7. **ReportRepository.java** – stores flagged messages in SQLite.
8. **ModelEvaluator.java / ModelExporter.java** – evaluation and export of the model for the browser.
9. **content.js, background.js, popup.js, speaksafe-model.js** – the browser extension and built-in model.

The system follows this workflow:

1. When the backend starts, the dataset is loaded, the TF-IDF vocabulary is learned and the Linear SVM is trained (in about one second).
2. The REST API waits for requests from the extension and the web pages.
3. The extension sends typed text and page messages, receives the result and warns, blurs or reports.
4. Flagged messages are stored in SQLite and shown on the moderator dashboard.

5.2 DATASET

The model is trained on a labelled dataset of **1520 messages** stored as a CSV file with two columns, label and text. The dataset contains 980 safe messages, 260 insults, 140 threats and

140 hate speech messages. It was built from sentence patterns that represent common forms of cyberbullying, together with about 250 hand-written everyday, news and anti-bullying sentences. Safe sentences that mention religions, nationalities and other groups in a positive or neutral way are included on purpose, so that the model learns that naming a group is not hate speech by itself. Swear words aimed at a person (for example “fuck off” or “you piece of shit”) are labelled as insults, while mild exclamations such as “damn, this traffic is terrible” are labelled safe. No racial or religious slurs are used in the dataset.

Category	Messages	Example
Safe	980	thanks for the notes, see you after class
Insult	260	you are such a pathetic loser / fuck off
Threat	140	I will beat you up after school
Hate speech	140	all refugees are criminals
Total	**1520**	

Table 5.1 – Composition of the Dataset

5.3 KEY FUNCTIONS (CODE EXPLANATION)

This section explains the main functions and logic implemented in each module.

1. TextPreprocessor.java (Preprocessing Module)

- **Purpose:** Converts raw text into a clean list of word stems.
- **Key Function:** process(String text) – performs the steps below in order:
  - Lower-casing and removal of links (http://…, www.…) and @mentions.
  - expandContractions() – “you’re” becomes “you are”, “I’ll” becomes “I will”, “can’t” becomes “can not”.
  - normalizeLeetspeak() – inside words, 0 → o, 1 → i, 3 → e, 4 → a, 5 → s, 7 → t, @ → a, $ → s and * is removed (so “l0s3r” becomes “loser” and “f*ck” becomes “fck”).
  - Collapsing repeated characters (“stuuupid” becomes “stupid”) and removing symbols and digits.
  - Tokenisation with OpenNLP’s SimpleTokenizer; a small SLANG table writes out masked and shortened swear words in full (“fck”, “fk” → “fuck”; “sht” → “shit”; “btch” → “bitch”); then stopword removal and stemming with OpenNLP’s PorterStemmer (“idiots” becomes “idiot”).
- **Implementation note:** Words such as “you”, “not”, “all” and “die” are deliberately kept, because they are important signals for bullying and threats.

2. TfIdfVectorizer.java (Feature Extraction Module)

- **Purpose:** Converts word stems into numbers.
- **Key Functions:**
  - fit(documents) – builds the vocabulary of single words and neighbouring word pairs (for example “kill_you”) and computes IDF = ln((1 + N) / (1 + df)) + 1.
  - transform(doc) – counts term frequencies, multiplies them by IDF and normalises the vector to unit length (L2 norm). Unknown words are ignored.
- **Implementation note:** Vectors are stored as maps of {feature index → weight} because most weights are zero.

3. SvmClassifier.java (Classification Module)

- **Purpose:** Linear Support Vector Machine with one separator per category (one-vs-rest).
- **Key Functions:**
  - train() – stochastic gradient descent on the hinge loss with L2 regularisation for 30 epochs, using a fixed random seed so that results are repeatable.
  - predictProbabilities() – converts each SVM distance into a value between 0 and 1 with a sigmoid. If no category is confident (total below 1), the remaining probability is given to SAFE, which avoids false alarms on unfamiliar text.

4. NaiveBayesClassifier.java (Classification Module)

- **Purpose:** Multinomial Naive Bayes used as a comparison model.
- **Key Functions:** train() learns log P(category) and log P(word | category) with smoothing; predictProbabilities() adds up the log-likelihoods and applies softmax.

5. ToxicityDetector.java (Categorization and Action Modules)

- **Purpose:** Turns model probabilities into a decision.
- **Key Function:** analyze(text)
  - score = round((1 − P(SAFE)) × 100).
  - If score < 50, the category is SAFE; otherwise it is the most likely of INSULT, THREAT and HATE_SPEECH.
  - decideAction(): SAFE → NONE; score ≥ 70 → BLUR; otherwise WARN. Both thresholds can be changed in application.properties.

6. DetectionController.java (Input Module – REST API)

Method	Endpoint	Purpose
GET	/api/health	Check whether the server is running
POST	/api/analyze	Analyse one message
POST	/api/analyze/batch	Analyse up to 50 messages from a page
POST	/api/report	A user reports a message
GET	/api/reports	List flagged and reported messages
DELETE	/api/reports/{id}	Dismiss a report
GET	/api/stats	Number of flagged messages per category
GET	/api/evaluate	Naive Bayes vs SVM evaluation

Table 5.2 – REST API Endpoints

7. ReportRepository.java (Database Module)

- **Purpose:** Saves, lists, deletes and counts flagged messages using Spring's JdbcTemplate.
- **Implementation note:** Uses INSERT OR IGNORE with a unique index, so automatic flags from repeated page scans do not create duplicates.

8. Chrome Extension (content.js, background.js, popup.js)

- **content.js** – listens to typing (with a 0.7 second pause) and shows a warning bubble; scans the page for message blocks with a TreeWalker, sends them in batches of 25, and blurs toxic ones with Show and Report buttons. A MutationObserver re-scans when new messages appear, for example in live chats.
- **background.js** – routes requests to the Java server, falls back to the built-in model if the server is unreachable, keeps counters and stores offline reports.
- **popup.js / reports.js** – settings switches, counters, quick test box and the list of flagged messages.

9. ModelExporter.java and speaksafe-model.js (Built-in Browser Model)

- ModelExporter saves the trained SVM (vocabulary, IDF weights, SVM weights, slang table and thresholds) to model.json (3,471 features, about 170 KB).
- speaksafe-model.js repeats every processing step in JavaScript, including a line-by-line port of OpenNLP's Porter stemmer.
- **Verification:** the JavaScript version produced exactly the same stems for 32,150 English words and exactly the same tokens, scores, categories and actions for 2,823 messages as the Java engine.

5.4 RESULTS AND DISCUSSION

5.4.1 Evaluation Method

The dataset is split into 80% training data and 20% test data, separately for each category (stratified split) and with a fixed random seed. Each model is trained on the training part and tested on the unseen test part. The following metrics are used, where TP, FP and FN are the true positives, false positives and false negatives of a category:

- **Accuracy** = correctly classified messages / all test messages
- **Precision** = TP / (TP + FP) – how many flagged messages were really of that category
- **Recall** = TP / (TP + FN) – how many messages of that category were found
- **F1-score** = 2 × Precision × Recall / (Precision + Recall)
- Macro averages are the mean over the four categories, so the smaller toxic categories count as much as SAFE.

5.4.2 Naive Bayes vs Linear SVM

Model	Accuracy	Precision	Recall	F1-score	ms / message
Naive Bayes	96.4%	96.7%	92.1%	94.2%	0.040
Linear SVM	95.7%	96.1%	91.2%	93.4%	0.039

Table 5.3 – Comparison of Naive Bayes and Linear SVM (20% test split)

Both models reach high accuracy on the test split (96.4% and 95.7%) and both analyse a message in well under a millisecond, which easily meets the real-time requirement. The per-category results and confusion matrix of the Linear SVM are shown in Tables 5.4 and 5.5.

Category	Precision	Recall	F1-score	Support
SAFE	95.6%	99.0%	97.2%	196
INSULT	96.1%	94.2%	95.1%	52
THREAT	100.0%	82.1%	90.2%	28
HATE SPEECH	92.6%	89.3%	90.9%	28

Table 5.4 – Per-Category Results of the Linear SVM

Actual \ Predicted	SAFE	INSULT	THREAT	HATE SPEECH
SAFE	194	0	0	2
INSULT	3	49	0	0
THREAT	3	2	23	0
HATE SPEECH	3	0	0	25

Table 5.5 – Confusion Matrix of the Linear SVM (rows = actual, columns = predicted)

5.4.3 Testing on Unseen Text

Because most of the dataset is generated from sentence patterns, the test-split results above are higher than what can be expected on real social media text. A fairer test was therefore carried out on hand-written text that the model never saw during training: short chat messages, article-style paragraphs and comments, the paragraphs of a news article about cyberbullying, and messages with swear words (including masked spellings such as “f*ck” and “sh*t”) together with mild exclamations.

Test set	Harmless texts left alone	Toxic texts caught
Short chat messages	44 / 44	25 / 26
Article-style paragraphs and comments	19 / 20	5 / 5
News article about cyberbullying	9 / 10	–
Profanity and masked swear words	4 / 4	16 / 16
Total	**76 / 78**	**46 / 47**

Table 5.6 – Results on Unseen Text (Linear SVM)

The mistakes made on unseen text were:

- “wait outside after class, you're getting beaten” – score 35, missed (shown as safe).
- “The charity provides free legal advice to refugees and migrants who are struggling to find housing and work.” – score 59, wrongly warned as hate speech.
- “Experts say threats such as "I know where you live" should always be reported to teachers and, where necess…” – score 98, wrongly blurred as threat.

With the same unseen text, Naive Bayes raised 2 false alarms and missed no toxic text, so the two models are within one message of each other. During development the ranking of the two models changed several times as the dataset grew: in earlier versions the SVM made clearly fewer false alarms on new text, while Naive Bayes scored higher on the test split. This shows that test-split accuracy alone can be misleading and that evaluation on realistic unseen text is important. The **Linear SVM is used as the default model** (it is also the model exported to the

browser extension), and Naive Bayes can be selected with the setting speaksafe.model=naive-bayes. A larger real-world dataset is needed to separate the two models reliably.

5.4.4 Improvements Made During Testing

Testing the extension on realistic pages revealed problems that the test-split accuracy did not show, and the system was improved as follows:

1. **Contractions:**
    - “I’ll” and “you’re” were split into meaningless tokens (“ll”, “re”) that the model linked to threats. Expanding contractions fixed false alarms such as “I’ll call you tomorrow”.
2. **Uncertain predictions:**
    - When no category was confident, the SVM probabilities were stretched until one toxic category looked certain. Giving the left-over probability to SAFE fixed messages such as “You” being flagged as a threat.
3. **Formal text:**
    - Words such as “society”, “all” and “their” appeared only in hate speech examples, so ordinary article paragraphs were blurred (4 of 10 paragraphs of a news article). Adding about 250 everyday, news and anti-bullying sentences reduced this to 1 of 10.
4. **Balanced group mentions:**
    - Each group name (religions, nationalities and others) was given the same number of positive or neutral safe sentences, so that mentioning a group does not by itself raise the score.
5. **Profanity:**
    - The first dataset contained no swear words, so “fuck off” was unknown to the model and scored as safe, and masked spellings such as “f*ck” were broken into “f ck” when symbols were removed. Directed profanity was added to the dataset as insults, mild exclamations as safe, and a small table now writes masked spellings out in full. All 16 profanity test messages are now caught while mild exclamations stay safe.

5.4.5 Output Screens

The following figures show the system in operation.

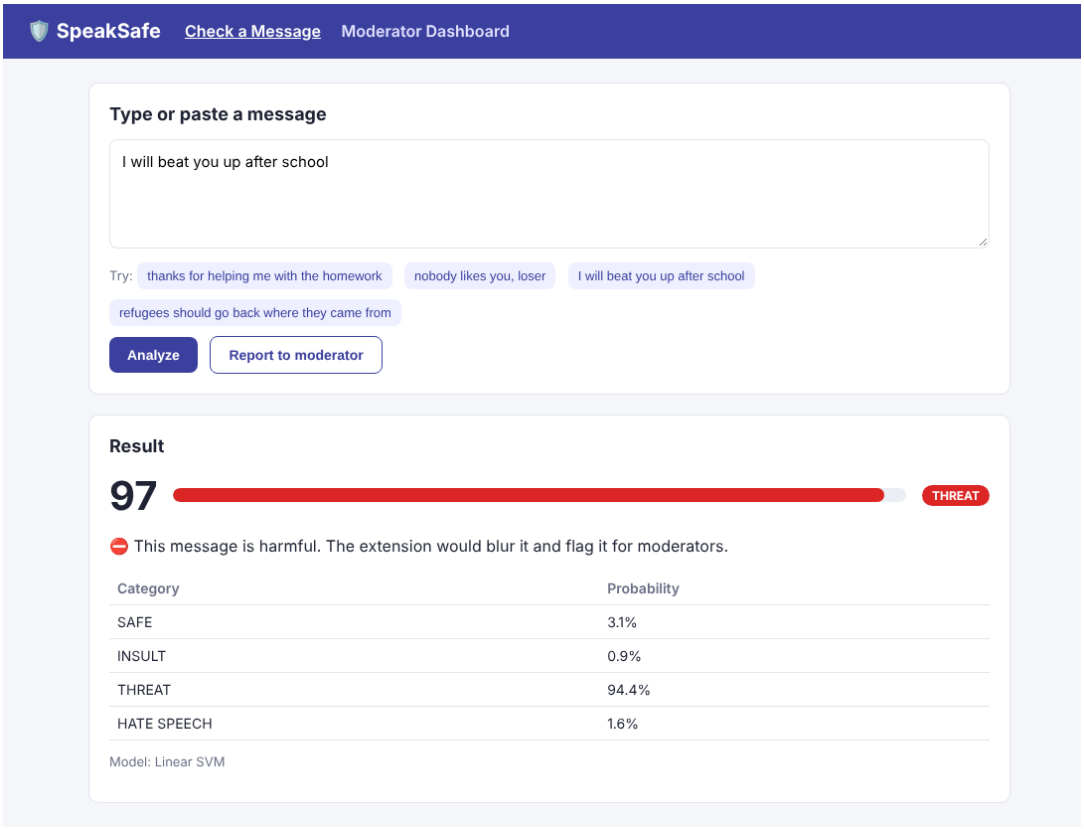


Fig 5.1 – Web Message Checker Showing Score, Category and Probabilities

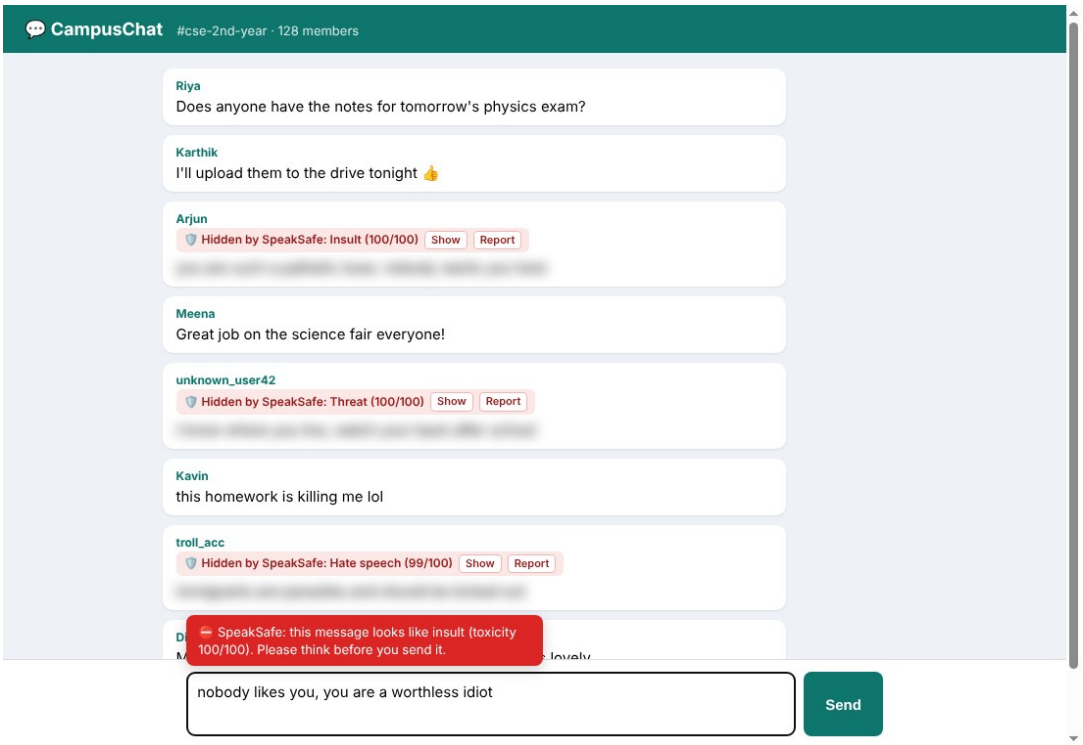


Fig 5.2 – Extension Warning the User While Typing in a Chat

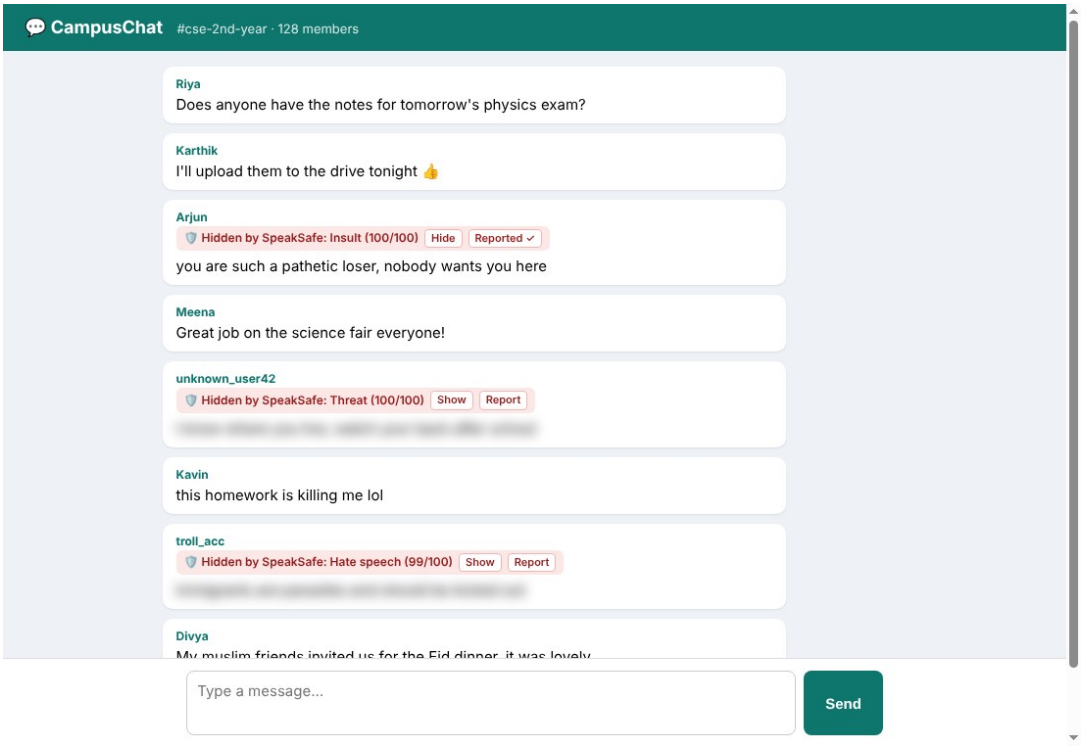


Fig 5.3 – Blurred Message Revealed with the Show Button and Reported

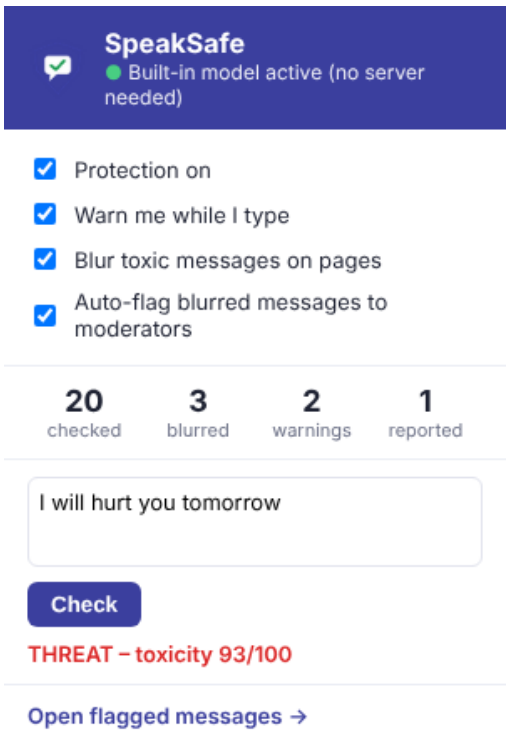


Fig 5.4 – Extension Popup Running the Built-in Model

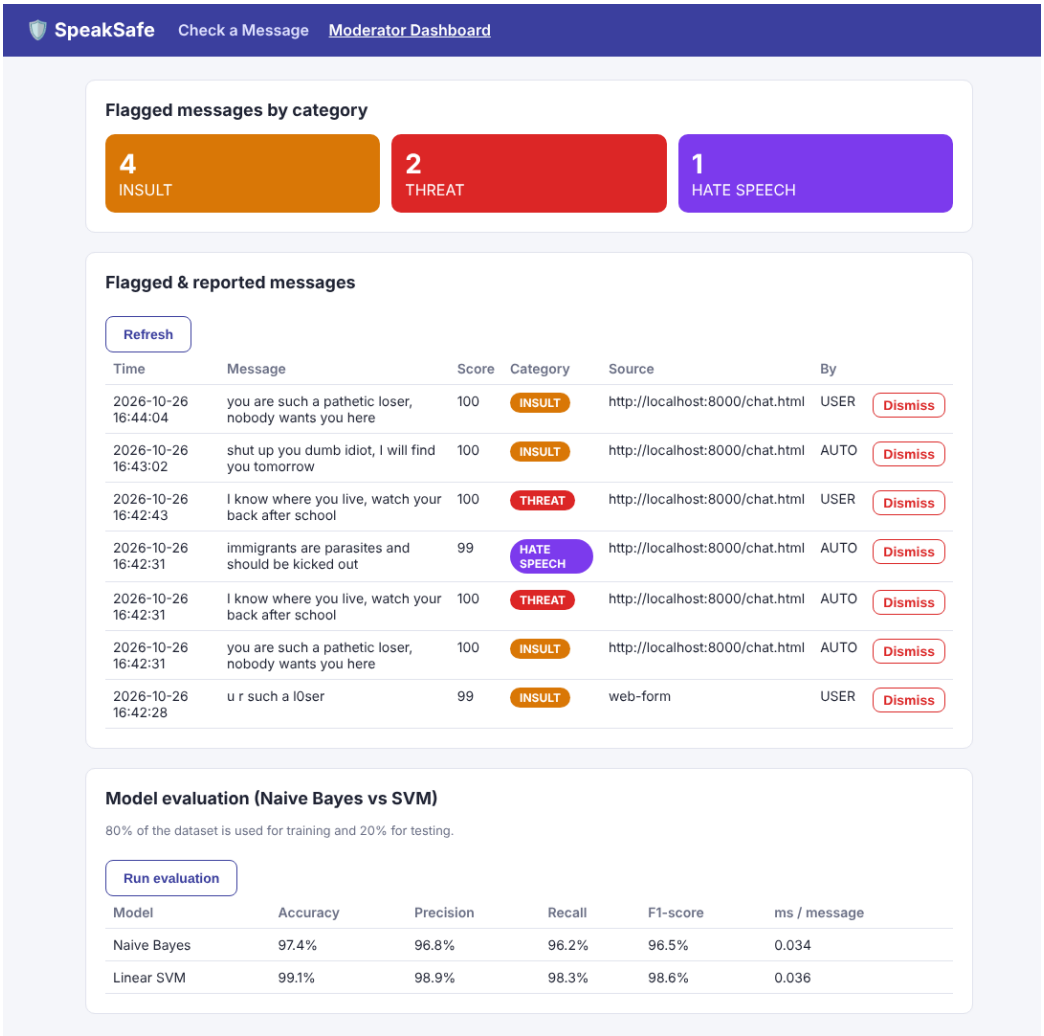


Fig 5.5 – Moderator Dashboard with Flagged Messages and Model Evaluation

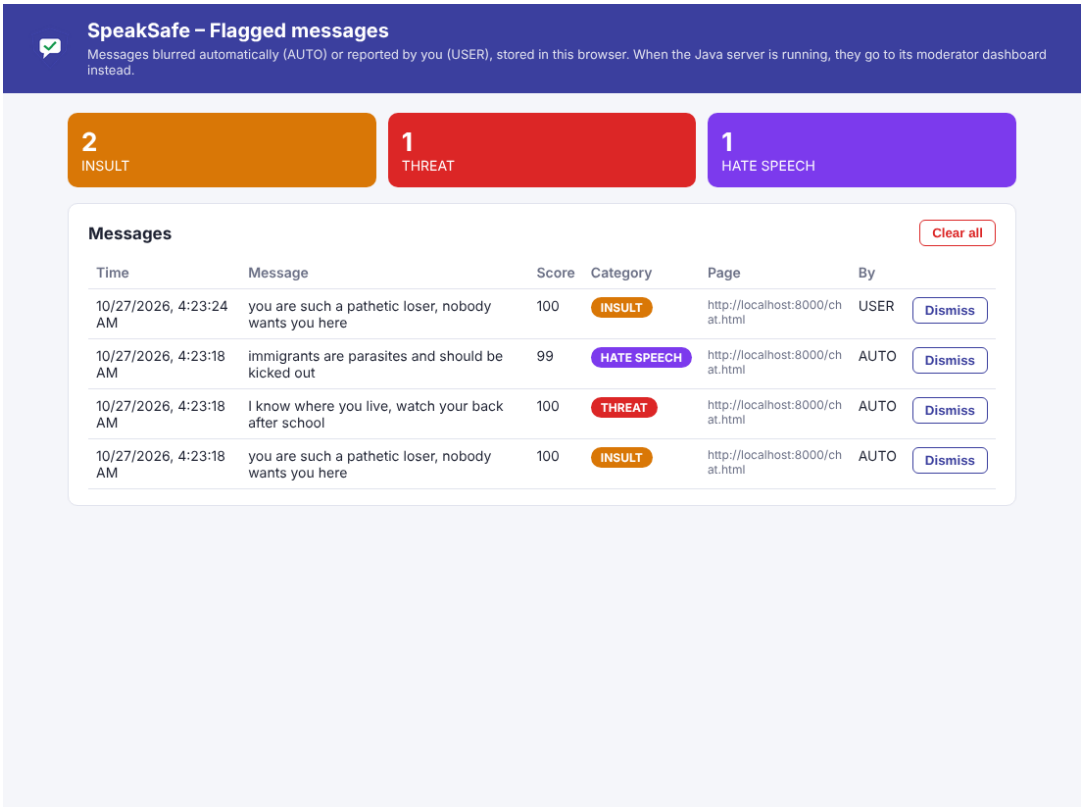


Fig 5.6 – Flagged Messages Page of the Extension (No Server Mode)

5.4.6 Limitations

- The training dataset is small and mostly generated from patterns, so accuracy on real social media text will be lower than the test-split results.
- Text that quotes abuse in order to discuss it (for example a news article quoting a threat) may still be flagged.
- Sarcasm, context across several messages and slang that is not in the dataset are hard for a bag-of-words model.
- Only English is supported; code-mixed text such as Tanglish is not handled.

CHAPTER 6
CONCLUSION & FUTURE WORK

6.1 CONCLUSION

The **SpeakSafe** project was developed to detect cyberbullying and hate speech in online text and to protect users in real time. It reads messages in the web browser, assigns each one a **toxicity score from 0 to 100** and a category (safe, insult, threat or hate speech), and automatically **warns, blurs or reports** harmful messages.

The system has been successfully implemented using **Java, Spring Boot, Apache OpenNLP, SQLite and a Chrome extension**, following a clear modular design:

- **TextPreprocessor** cleans and normalises text and handles leetspeak, masked swear words, repeated letters and contractions.
- **TfIdfVectorizer** turns text into TF-IDF features over words and word pairs.
- **SvmClassifier** and **NaiveBayesClassifier** predict category probabilities.
- **ToxicityDetector** computes the score, category and action.
- **DetectionController** and **ReportRepository** provide the REST API and storage for moderation.
- The **Chrome extension** warns while typing, blurs toxic messages and works even without the server.

The Linear SVM reached 95.7% accuracy on the test split while analysing a message in a fraction of a millisecond. On text it never saw during training, it left 76 of 78 harmless texts untouched and caught 46 of 47 toxic ones. The project shows that a lightweight, beginner-friendly Java system can provide useful real-time protection against online abuse on an ordinary computer, without GPUs or very large datasets.

Through this project, we gained practical experience in:

- Natural Language Processing and text preprocessing.
- Implementing machine learning algorithms (Naive Bayes and SVM) and evaluating them correctly.
- Building REST APIs with Spring Boot and storing data with SQLite.
- Developing a Manifest V3 browser extension with HTML, CSS and JavaScript.
- Testing a system on realistic data and improving it based on the errors found.

6.2 FUTURE ENHANCEMENTS

Although the current implementation meets the objectives of the project, there is significant potential for improvement:

1. **Larger real-world datasets:**
    - Train on public datasets such as the Jigsaw Toxic Comment Classification dataset to improve accuracy on real social media text.
2. **Deep learning and transformer models:**
    - Add LSTM or lightweight transformer models (for example DistilBERT) through the existing Classifier interface for better understanding of context and sarcasm.
3. **Multilingual support:**
    - Support Tamil, Hindi and code-mixed text such as Tanglish, which is common among students in India.
4. **Context awareness:**
    - Consider previous messages in a conversation, and detect repeated targeting of the same person.
5. **Explainability:**
    - Highlight the words that caused a message to be flagged, so that users understand the warning.
6. **Mobile support:**
    - Provide an Android keyboard or app version for messaging applications.
7. **Chrome Web Store publication:**
    - Publish the extension so that anyone can install it with one click.
8. **Feedback loop:**
    - Let moderators mark false alarms and missed messages, and use this feedback to retrain the model.

APPENDIX 1
SOURCE CODE

The appendix includes the source code of the main modules of SpeakSafe. The complete project, including the web pages and tests, is available in the project repository.

FILE STRUCTURE

```
SpeakSafe
├── backend
│   ├── pom.xml
│   └── src/main
│       ├── java/com/speaksafe
│       │   ├── SpeakSafeApplication.java
│       │   ├── model       (Category, Action, AnalysisResult, Report)
│       │   ├── nlp         (TextPreprocessor, TfIdfVectorizer)
│       │   ├── ml          (Classifier, NaiveBayesClassifier, SvmClassifier,
│       │   │                DatasetLoader, DetectionPipeline,
│       │   │                ModelEvaluator, ModelExporter)
│       │   ├── service     (ToxicityDetector, ReportRepository)
│       │   └── controller  (DetectionController)
│       └── resources
│           ├── application.properties
│           ├── schema.sql
│           ├── dataset/speaksafe_dataset.csv
│           └── static      (index.html, dashboard.html, style.css)
└── extension
    ├── manifest.json
    ├── background.js
    ├── content.js, content.css
    ├── speaksafe-model.js, model.json
    ├── popup.html, popup.js, popup.css
    ├── reports.html, reports.js, reports.css
    └── icons
```

MODULE 1 – DATA MODELS

Category.java

```
package com.speaksafe.model;

/** The labels a message can receive. */
public enum Category {
    SAFE,
    INSULT,
    THREAT,
    HATE_SPEECH;

    /** Reads a label from the dataset, e.g. "hate_speech" or "HATE SPEECH". */
    public static Category fromLabel(String label) {
        return Category.valueOf(label.trim().toUpperCase().replace(' ', '_'));
    }
}
```

Action.java

```
package com.speaksafe.model;

/** What the browser extension should do with a message. */
public enum Action {
    NONE,   // safe message, do nothing
    WARN,   // show a warning to the user
    BLUR    // hide the message behind a blur and flag it for moderators
}
```

AnalysisResult.java

```
package com.speaksafe.model;

import java.util.Map;

/**
 * Output of the detection engine for one message.
 *
 * @param text          the original message
 * @param score         toxicity score from 0 (safe) to 100 (very toxic)
 * @param category      SAFE, INSULT, THREAT or HATE_SPEECH
 * @param action        NONE, WARN or BLUR
 * @param probabilities probability of each category (0.0 - 1.0)
 * @param model         name of the model that produced the result
 */
public record AnalysisResult(String text,
                             int score,
                             Category category,
                             Action action,
                             Map<Category, Double> probabilities,
                             String model) {
}
```

Report.java

```
package com.speaksafe.model;

/** A flagged or reported message stored in the database. */
public record Report(long id,
                     String text,
                     int score,
                     Category category,
                     String sourceUrl,
                     String reportedBy,
                     String createdAt) {
}
```

MODULE 2 – PREPROCESSING MODULE

TextPreprocessor.java

```
package com.speaksafe.nlp;
```

```
import opennlp.tools.stemmer.PorterStemmer;
import opennlp.tools.tokenize.SimpleTokenizer;

import java.util.ArrayList;
import java.util.List;
import java.util.Map;
import java.util.Set;

/**
 * MODULE: Preprocessing.
 * Cleans raw text and turns it into a list of word stems, for example:
 *   "You are SUCH an 1d10t!!! http://x.com"  ->  [you, such, idiot]
 */
public class TextPreprocessor {

    // Common words that carry no meaning. Words like "you", "not", "all" and "die"
    // are kept on purpose because they matter a lot for bullying and threats.
    private static final Set<String> STOPWORDS = Set.of(
            "a", "an", "the", "is", "am", "are", "was", "were", "be", "been", "being",
            "and", "or", "but", "if", "of", "at", "by", "for", "with", "about", "to",
            "from", "in", "on", "this", "that", "these", "those", "it", "its", "so",
            "as", "do", "does", "did", "have", "has", "had", "just", "very", "can",
            "will", "would", "should", "could", "i", "me", "my", "we", "our", "us",
            "then", "than", "there", "here", "what", "which", "who", "when", "where",
            "how", "too", "also", "such", "some", "any", "into", "up", "out", "over");

    // Common disguised or shortened spellings of swear words, written out in full
    // so the model recognises them (e.g. "f*ck" -> "fck" -> "fuck").
    private static final Map<String, String> SLANG = Map.ofEntries(
            Map.entry("fck", "fuck"), Map.entry("fk", "fuck"), Map.entry("fuk", "fuck"),
            Map.entry("fuq", "fuck"), Map.entry("fuc", "fuck"), Map.entry("fcuk",
"fuck"),
            Map.entry("phuck", "fuck"), Map.entry("fking", "fucking"),
Map.entry("fcking", "fucking"),
            Map.entry("fuking", "fucking"), Map.entry("fkin", "fucking"),
Map.entry("fckin", "fucking"),
            Map.entry("fkn", "fucking"), Map.entry("fcker", "fucker"), Map.entry("fker",
"fucker"),
            Map.entry("sht", "shit"), Map.entry("shyt", "shit"), Map.entry("btch",
"bitch"),
            Map.entry("biatch", "bitch"), Map.entry("bich", "bitch"), Map.entry("ahole",
"asshole"),
            Map.entry("azzhole", "asshole"), Map.entry("mofo", "motherfucker"),
Map.entry("bstrd", "bastard"),
            Map.entry("dck", "dick"), Map.entry("idiotic", "idiot"));

    public static Set<String> stopwords() {
        return STOPWORDS;
    }

    public static Map<String, String> slang() {
        return SLANG;
    }

    private final SimpleTokenizer tokenizer = SimpleTokenizer.INSTANCE;
    private final PorterStemmer stemmer = new PorterStemmer();

    public List<String> process(String text) {
```

```
        if (text == null) {
            return List.of();
        }
        String clean = text.toLowerCase();
        clean = clean.replaceAll("https?://\\S+|www\\.\\S+", " ");   // remove links
        clean = clean.replaceAll("@\\w+", " ");                      // remove @mentions
        clean = expandContractions(clean);                           // you're -> you are
        clean = normalizeLeetspeak(clean);                           // 1d10t -> idiot
        clean = clean.replaceAll("(.)\\1{2,}", "$1");                // stuuupid ->
stupid
        clean = clean.replaceAll("[^a-z\\s]", " ");                  // remove symbols
and digits

        List<String> tokens = new ArrayList<>();
        for (String token : tokenizer.tokenize(clean)) {
            token = SLANG.getOrDefault(token, token);              // f*ck / fck -> fuck
            if (token.length() > 1 && !STOPWORDS.contains(token)) {
                tokens.add(stemmer.stem(token));
            }
        }
        return tokens;
    }

    /** Writes short forms out in full, so "I'll" and "you're" become real words instead
of "ll" and "re". */
    private String expandContractions(String text) {
        return text.replace('\u2019', '\'')                       // curly apostrophe ->
straight
                   .replaceAll("\\bcan't\\b", "can not")
                   .replaceAll("\\bwon't\\b", "will not")
                   .replaceAll("n't\\b", " not")
                   .replaceAll("'re\\b", " are")
                   .replaceAll("'ll\\b", " will")
                   .replaceAll("'m\\b", " am")
                   .replaceAll("'ve\\b", " have")
                   .replaceAll("'d\\b", " would")
                   .replaceAll("'s\\b", "");
    }

    /** Replaces digits and symbols used to hide bad words, but only inside words that
contain letters. */
    private String normalizeLeetspeak(String text) {
        StringBuilder result = new StringBuilder();
        for (String word : text.split("\\s+")) {
            if (word.matches(".*[a-z].*")) {
                word = word.replace('0', 'o').replace('1', 'i').replace('3', 'e')
                           .replace('4', 'a').replace('5', 's').replace('7', 't')
                           .replace('@', 'a').replace('$', 's')
                           .replace("*", "");                                 // f*ck ->
fck
            }
            result.append(word).append(' ');
        }
        return result.toString();
    }
}
```

MODULE 3 – FEATURE EXTRACTION MODULE

TfIdfVectorizer.java

```
package com.speaksafe.nlp;

import java.util.ArrayList;
import java.util.HashMap;
import java.util.HashSet;
import java.util.List;
import java.util.Map;

/**
 * MODULE: Feature Extraction.
 * Converts a list of words into numbers using TF-IDF.
 * It uses single words ("kill") and word pairs ("kill you") as features.
 * Vectors are stored as a map of {feature index -> weight}, because most weights are
zero.
 */
public class TfIdfVectorizer {

    private final Map<String, Integer> vocabulary = new HashMap<>();
    private double[] idf;

    /** Learns the vocabulary and IDF weights from the training documents. */
    public void fit(List<List<String>> documents) {
        Map<String, Integer> documentFrequency = new HashMap<>();
        for (List<String> doc : documents) {
            for (String term : new HashSet<>(terms(doc))) {
                documentFrequency.merge(term, 1, Integer::sum);
            }
        }
        for (String term : documentFrequency.keySet()) {
            vocabulary.put(term, vocabulary.size());
        }
        int n = documents.size();
        idf = new double[vocabulary.size()];
        for (Map.Entry<String, Integer> entry : vocabulary.entrySet()) {
            int df = documentFrequency.get(entry.getKey());
            idf[entry.getValue()] = Math.log((1.0 + n) / (1.0 + df)) + 1.0;
        }
    }

    /** Turns one document into a normalized TF-IDF vector. Unknown words are ignored.
*/
    public Map<Integer, Double> transform(List<String> doc) {
        Map<Integer, Double> vector = new HashMap<>();
        for (String term : terms(doc)) {
            Integer index = vocabulary.get(term);
            if (index != null) {
                vector.merge(index, 1.0, Double::sum);         // term frequency
            }
        }
        double length = 0;
        for (Map.Entry<Integer, Double> entry : vector.entrySet()) {
            double weight = entry.getValue() * idf[entry.getKey()];
            entry.setValue(weight);
```

```
            length += weight * weight;
        }
        double norm = Math.sqrt(length);
        if (norm > 0) {
            vector.replaceAll((index, weight) -> weight / norm);
        }
        return vector;
    }

    public int size() {
        return vocabulary.size();
    }

    /** Word (or word pair) -> feature index. Used when exporting the model. */
    public Map<String, Integer> vocabulary() {
        return vocabulary;
    }

    /** IDF weight of every feature. Used when exporting the model. */
    public double[] idf() {
        return idf;
    }

    /** Single words plus pairs of neighbouring words. */
    private List<String> terms(List<String> words) {
        List<String> terms = new ArrayList<>(words);
        for (int i = 0; i + 1 < words.size(); i++) {
            terms.add(words.get(i) + "_" + words.get(i + 1));
        }
        return terms;
    }
}
```

MODULE 4 – CLASSIFICATION MODULE

Classifier.java

```
package com.speaksafe.ml;

import com.speaksafe.model.Category;

import java.util.List;
import java.util.Map;

/** Common interface so the Naive Bayes and SVM models can be swapped and compared. */
public interface Classifier {

    /** Learns from TF-IDF vectors and their correct labels. */
    void train(List<Map<Integer, Double>> vectors, List<Category> labels, int
featureCount);

    /** Returns the probability (0.0 - 1.0) of each category for one TF-IDF vector. */
    Map<Category, Double> predictProbabilities(Map<Integer, Double> vector);

    String name();
}
```

NaiveBayesClassifier.java

```
package com.speaksafe.ml;

import com.speaksafe.model.Category;

import java.util.EnumMap;
import java.util.List;
import java.util.Map;

/**
 * MODULE: Classification (model 1 of 2).
 * Multinomial Naive Bayes: learns how often each word appears in each category,
 * then picks the category that best explains the words of a new message.
 */
public class NaiveBayesClassifier implements Classifier {

    private static final double SMOOTHING = 0.1;   // stops unseen words from giving
zero probability

    private final Category[] categories = Category.values();
    private double[] logPrior;                     // log P(category)
    private double[][] logLikelihood;              // log P(word | category)

    @Override
    public void train(List<Map<Integer, Double>> vectors, List<Category> labels, int
featureCount) {
        int k = categories.length;
        double[] docsPerClass = new double[k];
        double[][] wordWeight = new double[k][featureCount];
        double[] totalWeight = new double[k];

        for (int i = 0; i < vectors.size(); i++) {
            int c = labels.get(i).ordinal();
            docsPerClass[c]++;
            for (Map.Entry<Integer, Double> entry : vectors.get(i).entrySet()) {
                wordWeight[c][entry.getKey()] += entry.getValue();
                totalWeight[c] += entry.getValue();
            }
        }

        logPrior = new double[k];
        logLikelihood = new double[k][featureCount];
        for (int c = 0; c < k; c++) {
            logPrior[c] = Math.log(docsPerClass[c] / vectors.size());
            double denominator = totalWeight[c] + SMOOTHING * featureCount;
            for (int f = 0; f < featureCount; f++) {
                logLikelihood[c][f] = Math.log((wordWeight[c][f] + SMOOTHING) /
denominator);
            }
        }
    }

    @Override
    public Map<Category, Double> predictProbabilities(Map<Integer, Double> vector) {
        double[] logScores = new double[categories.length];
        for (int c = 0; c < categories.length; c++) {
```

```
            logScores[c] = logPrior[c];
            for (Map.Entry<Integer, Double> entry : vector.entrySet()) {
                logScores[c] += entry.getValue() * logLikelihood[c][entry.getKey()];
            }
        }
        return toProbabilities(logScores);
    }

    /** Softmax: converts log scores into probabilities that add up to 1. */
    private Map<Category, Double> toProbabilities(double[] logScores) {
        double max = Double.NEGATIVE_INFINITY;
        for (double s : logScores) max = Math.max(max, s);
        double sum = 0;
        double[] exp = new double[logScores.length];
        for (int c = 0; c < logScores.length; c++) {
            exp[c] = Math.exp(logScores[c] - max);
            sum += exp[c];
        }
        Map<Category, Double> probabilities = new EnumMap<>(Category.class);
        for (int c = 0; c < categories.length; c++) {
            probabilities.put(categories[c], exp[c] / sum);
        }
        return probabilities;
    }

    @Override
    public String name() {
        return "Naive Bayes";
    }
}
```

SvmClassifier.java

```
package com.speaksafe.ml;

import com.speaksafe.model.Category;

import java.util.ArrayList;
import java.util.Collections;
import java.util.EnumMap;
import java.util.List;
import java.util.Map;
import java.util.Random;

/**
 * MODULE: Classification (model 2 of 2).
 * Linear Support Vector Machine (SVM), "one-vs-rest":
 * one straight-line separator is trained per category (e.g. THREAT vs everything else).
 * Training uses stochastic gradient descent on the hinge loss.
 */
public class SvmClassifier implements Classifier {

    private static final int EPOCHS = 30;
    private static final double LEARNING_RATE = 0.1;
    private static final double REGULARIZATION = 0.0001;

    private final Category[] categories = Category.values();
    private double[][] weights;   // one weight per feature, per category
```

```
    private double[] bias;

    @Override
    public void train(List<Map<Integer, Double>> vectors, List<Category> labels, int
featureCount) {
        weights = new double[categories.length][featureCount];
        bias = new double[categories.length];

        List<Integer> order = new ArrayList<>();
        for (int i = 0; i < vectors.size(); i++) order.add(i);
        Random random = new Random(42);   // fixed seed -> same result every run

        for (int epoch = 0; epoch < EPOCHS; epoch++) {
            Collections.shuffle(order, random);
            for (int i : order) {
                Map<Integer, Double> x = vectors.get(i);
                for (int c = 0; c < categories.length; c++) {
                    double y = labels.get(i).ordinal() == c ? 1.0 : -1.0;
                    double margin = y * score(c, x);
                    shrinkWeights(c, x);
                    if (margin < 1) {   // wrong or not confident enough -> move the
line
                        for (Map.Entry<Integer, Double> e : x.entrySet()) {
                            weights[c][e.getKey()] += LEARNING_RATE * y * e.getValue();
                        }
                        bias[c] += LEARNING_RATE * y;
                    }
                }
            }
        }
    }

    /** L2 regularization, applied only to the features present in this example (keeps
training fast). */
    private void shrinkWeights(int c, Map<Integer, Double> x) {
        for (int f : x.keySet()) {
            weights[c][f] *= (1 - LEARNING_RATE * REGULARIZATION);
        }
    }

    private double score(int c, Map<Integer, Double> x) {
        double s = bias[c];
        for (Map.Entry<Integer, Double> e : x.entrySet()) {
            s += weights[c][e.getKey()] * e.getValue();
        }
        return s;
    }

    @Override
    public Map<Category, Double> predictProbabilities(Map<Integer, Double> vector) {
        // An SVM gives distances, not probabilities. We squash each distance into 0..1
with a sigmoid.
        double[] p = new double[categories.length];
        double sum = 0;
        for (int c = 0; c < categories.length; c++) {
            p[c] = 1.0 / (1.0 + Math.exp(-3.0 * score(c, vector)));
            sum += p[c];
        }
```

```
        // If no category is confident (total below 1), the message is unlike anything
toxic
        // the model has seen, so the left-over probability goes to SAFE.
        int safe = Category.SAFE.ordinal();
        if (sum < 1.0) {
            p[safe] += 1.0 - sum;
            sum = 1.0;
        }
        Map<Category, Double> probabilities = new EnumMap<>(Category.class);
        for (int c = 0; c < categories.length; c++) {
            probabilities.put(categories[c], p[c] / sum);
        }
        return probabilities;
    }

    public double[][] weights() {
        return weights;
    }

    public double[] bias() {
        return bias;
    }

    @Override
    public String name() {
        return "Linear SVM";
    }
}
```

MODULE 5 – TRAINING, EVALUATION AND EXPORT

DatasetLoader.java

```
package com.speaksafe.ml;

import com.speaksafe.model.Category;

import java.io.BufferedReader;
import java.io.IOException;
import java.io.InputStream;
import java.io.InputStreamReader;
import java.io.UncheckedIOException;
import java.nio.charset.StandardCharsets;
import java.util.ArrayList;
import java.util.List;

/**
 * Loads the labelled dataset (CSV file with the columns: label,text).
 * Example line:   insult,"you are such a loser"
 */
public class DatasetLoader {

    public static final String DEFAULT_DATASET = "/dataset/speaksafe_dataset.csv";

    /** One labelled example. */
    public record Sample(String text, Category label) { }
```

```
    public static List<Sample> load(String resourcePath) {
        List<Sample> samples = new ArrayList<>();
        InputStream in = DatasetLoader.class.getResourceAsStream(resourcePath);
        if (in == null) {
            throw new IllegalStateException("Dataset not found: " + resourcePath);
        }
        try (BufferedReader reader = new BufferedReader(new InputStreamReader(in,
StandardCharsets.UTF_8))) {
            String line = reader.readLine();   // skip header
            while ((line = reader.readLine()) != null) {
                if (line.isBlank()) continue;
                int comma = line.indexOf(',');
                String label = line.substring(0, comma);
                String text = line.substring(comma + 1).trim();
                if (text.startsWith("\"") && text.endsWith("\"")) {
                    text = text.substring(1, text.length() - 1).replace("\"\"", "\"");
                }
                samples.add(new Sample(text, Category.fromLabel(label)));
            }
        } catch (IOException e) {
            throw new UncheckedIOException(e);
        }
        return samples;
    }
}
```

DetectionPipeline.java

```
package com.speaksafe.ml;

import com.speaksafe.model.Category;
import com.speaksafe.nlp.TextPreprocessor;
import com.speaksafe.nlp.TfIdfVectorizer;

import java.util.ArrayList;
import java.util.EnumMap;
import java.util.List;
import java.util.Map;

/**
 * Connects the modules in order:
 *   raw text -> Preprocessing -> Feature Extraction (TF-IDF) -> Classification ->
probabilities
 */
public class DetectionPipeline {

    private final TextPreprocessor preprocessor = new TextPreprocessor();
    private final TfIdfVectorizer vectorizer = new TfIdfVectorizer();
    private final Classifier classifier;

    public DetectionPipeline(Classifier classifier) {
        this.classifier = classifier;
    }

    public void train(List<DatasetLoader.Sample> samples) {
        List<List<String>> documents = new ArrayList<>();
        List<Category> labels = new ArrayList<>();
```

```
        for (DatasetLoader.Sample sample : samples) {
            documents.add(preprocessor.process(sample.text()));
            labels.add(sample.label());
        }
        vectorizer.fit(documents);

        List<Map<Integer, Double>> vectors = new ArrayList<>();
        for (List<String> doc : documents) {
            vectors.add(vectorizer.transform(doc));
        }
        classifier.train(vectors, labels, vectorizer.size());
    }

    public Map<Category, Double> predict(String text) {
        Map<Integer, Double> vector = vectorizer.transform(preprocessor.process(text));
        if (vector.isEmpty()) {
            // No known words (e.g. "ok", emojis, a new language): treat as safe.
            Map<Category, Double> safe = new EnumMap<>(Category.class);
            for (Category c : Category.values()) safe.put(c, c == Category.SAFE ? 1.0 :
0.0);
            return safe;
        }
        return classifier.predictProbabilities(vector);
    }

    /** The category with the highest probability. */
    public Category predictCategory(String text) {
        Map<Category, Double> probabilities = predict(text);
        Category best = Category.SAFE;
        for (Map.Entry<Category, Double> e : probabilities.entrySet()) {
            if (e.getValue() > probabilities.get(best)) best = e.getKey();
        }
        return best;
    }

    public TfIdfVectorizer vectorizer() {
        return vectorizer;
    }

    public Classifier classifier() {
        return classifier;
    }

    public String modelName() {
        return classifier.name();
    }
}
```

ModelEvaluator.java

```
package com.speaksafe.ml;

import com.speaksafe.model.Category;

import java.util.ArrayList;
import java.util.Collections;
import java.util.EnumMap;
import java.util.List;
```

```
import java.util.Map;
import java.util.Random;
import java.util.function.Supplier;

/**
 * Evaluation: splits the dataset into 80% training and 20% testing data,
 * trains a model, and measures accuracy, precision, recall, F1-score and speed.
 */
public class ModelEvaluator {

    /** Scores for one category. */
    public record ClassMetrics(double precision, double recall, double f1, int support)
{ }

    /** Full result for one model. */
    public record EvaluationResult(String model,
                                   int trainSize,
                                   int testSize,
                                   double accuracy,
                                   double macroPrecision,
                                   double macroRecall,
                                   double macroF1,
                                   double avgMillisPerMessage,
                                   Map<Category, ClassMetrics> perCategory,
                                   int[][] confusionMatrix) { }

    public static EvaluationResult evaluate(Supplier<Classifier> modelFactory,
List<DatasetLoader.Sample> data) {
        List<DatasetLoader.Sample> train = new ArrayList<>();
        List<DatasetLoader.Sample> test = new ArrayList<>();
        splitStratified(data, train, test);

        DetectionPipeline pipeline = new DetectionPipeline(modelFactory.get());
        pipeline.train(train);

        int k = Category.values().length;
        int[][] confusion = new int[k][k];   // rows = actual, columns = predicted
        long start = System.nanoTime();
        for (DatasetLoader.Sample sample : test) {
            Category predicted = pipeline.predictCategory(sample.text());
            confusion[sample.label().ordinal()][predicted.ordinal()]++;
        }
        double avgMillis = (System.nanoTime() - start) / 1_000_000.0 / test.size();

        int correct = 0;
        double sumP = 0, sumR = 0, sumF = 0;
        Map<Category, ClassMetrics> perCategory = new EnumMap<>(Category.class);
        for (Category c : Category.values()) {
            int i = c.ordinal();
            int truePositive = confusion[i][i];
            int predictedAsC = 0, actuallyC = 0;
            for (int j = 0; j < k; j++) {
                predictedAsC += confusion[j][i];
                actuallyC += confusion[i][j];
            }
            correct += truePositive;
            double precision = predictedAsC == 0 ? 0 : (double) truePositive /
predictedAsC;
```

```
            double recall = actuallyC == 0 ? 0 : (double) truePositive / actuallyC;
            double f1 = precision + recall == 0 ? 0 : 2 * precision * recall /
(precision + recall);
            perCategory.put(c, new ClassMetrics(precision, recall, f1, actuallyC));
            sumP += precision;
            sumR += recall;
            sumF += f1;
        }
        return new EvaluationResult(pipeline.modelName(), train.size(), test.size(),
                (double) correct / test.size(), sumP / k, sumR / k, sumF / k,
                avgMillis, perCategory, confusion);
    }

    /** Puts 80% of every category into training and 20% into testing, the same way
every run. */
    private static void splitStratified(List<DatasetLoader.Sample> data,
                                        List<DatasetLoader.Sample> train,
                                        List<DatasetLoader.Sample> test) {
        Random random = new Random(7);
        for (Category c : Category.values()) {
            List<DatasetLoader.Sample> group = new ArrayList<>();
            for (DatasetLoader.Sample s : data) {
                if (s.label() == c) group.add(s);
            }
            Collections.shuffle(group, random);
            int cut = (int) Math.round(group.size() * 0.8);
            train.addAll(group.subList(0, cut));
            test.addAll(group.subList(cut, group.size()));
        }
    }

    /** Runs both models and prints a comparison table (used with --evaluate). */
    public static void printComparison() {
        List<DatasetLoader.Sample> data =
DatasetLoader.load(DatasetLoader.DEFAULT_DATASET);
        System.out.println("\n================ SpeakSafe Model Evaluation
================");
        System.out.println("Dataset size: " + data.size() + " messages (80% train / 20%
test)\n");

        List<EvaluationResult> results = List.of(
                evaluate(NaiveBayesClassifier::new, data),
                evaluate(SvmClassifier::new, data));

        System.out.printf("%-14s %9s %10s %8s %9s %12s%n",
                "Model", "Accuracy", "Precision", "Recall", "F1-score", "ms/message");
        for (EvaluationResult r : results) {
            System.out.printf("%-14s %8.1f%% %9.1f%% %7.1f%% %8.1f%% %12.3f%n",
r.model(),
                    r.accuracy() * 100, r.macroPrecision() * 100, r.macroRecall() * 100,
                    r.macroF1() * 100, r.avgMillisPerMessage());
        }

        for (EvaluationResult r : results) {
            System.out.println("\n--- " + r.model() + ": per category ---");
            System.out.printf("%-12s %10s %8s %9s %8s%n", "Category", "Precision",
"Recall", "F1-score", "Support");
            for (Map.Entry<Category, ClassMetrics> e : r.perCategory().entrySet()) {
```

```
                ClassMetrics m = e.getValue();
                System.out.printf("%-12s %9.1f%% %7.1f%% %8.1f%% %8d%n", e.getKey(),
                        m.precision() * 100, m.recall() * 100, m.f1() * 100,
m.support());
            }
            System.out.println("Confusion matrix (rows = actual, columns =
predicted):");
            System.out.printf("%-12s", "");
            for (Category c : Category.values()) System.out.printf("%12s", c);
            System.out.println();
            for (Category actual : Category.values()) {
                System.out.printf("%-12s", actual);
                for (Category predicted : Category.values()) {
                    System.out.printf("%12d", r.confusionMatrix()[actual.ordinal()]
[predicted.ordinal()]);
                }
                System.out.println();
            }
        }

System.out.println("============================================================\n");
    }
}
```

ModelExporter.java

```
package com.speaksafe.ml;

import com.fasterxml.jackson.databind.ObjectMapper;
import com.speaksafe.model.Category;
import com.speaksafe.nlp.TextPreprocessor;

import java.io.File;
import java.io.IOException;
import java.util.ArrayList;
import java.util.LinkedHashMap;
import java.util.List;
import java.util.Map;
import java.util.TreeSet;

/**
 * Exports the trained Linear SVM as a JSON file, so the same model can also run
 * inside the browser extension and the web demo without the Java server.
 *
 *   java -jar speaksafe.jar --export-model ../extension/model.json
 */
public class ModelExporter {

    public static void export(String path, int warnThreshold, int blurThreshold) throws
IOException {
        List<DatasetLoader.Sample> dataset =
DatasetLoader.load(DatasetLoader.DEFAULT_DATASET);
        SvmClassifier svm = new SvmClassifier();
        DetectionPipeline pipeline = new DetectionPipeline(svm);
        pipeline.train(dataset);

        // Put the features in index order so the arrays below line up with "terms".
        Map<String, Integer> vocabulary = pipeline.vectorizer().vocabulary();
```

```
        String[] terms = new String[vocabulary.size()];
        vocabulary.forEach((term, index) -> terms[index] = term);

        Map<String, Object> model = new LinkedHashMap<>();
        model.put("name", "SpeakSafe Linear SVM");
        model.put("trainedOn", dataset.size());
        model.put("warnThreshold", warnThreshold);
        model.put("blurThreshold", blurThreshold);
        model.put("categories", Category.values());
        model.put("stopwords", new TreeSet<>(TextPreprocessor.stopwords()));
        model.put("slang", new java.util.TreeMap<>(TextPreprocessor.slang()));
        model.put("terms", terms);
        model.put("idf", round(pipeline.vectorizer().idf()));
        List<double[]> weights = new ArrayList<>();
        for (double[] row : svm.weights()) weights.add(round(row));
        model.put("weights", weights);
        model.put("bias", round(svm.bias()));

        new ObjectMapper().writeValue(new File(path), model);
        System.out.println("Model exported to " + path + " (" + terms.length + "
features)");
    }

    /** Keeps 5 decimal places, which makes the file much smaller without changing
results. */
    private static double[] round(double[] values) {
        double[] rounded = new double[values.length];
        for (int i = 0; i < values.length; i++) {
            rounded[i] = Math.round(values[i] * 100000) / 100000.0;
        }
        return rounded;
    }
}
```

MODULE 6 – CATEGORIZATION AND ACTION MODULE

ToxicityDetector.java

```
package com.speaksafe.service;

import com.speaksafe.ml.DatasetLoader;
import com.speaksafe.ml.DetectionPipeline;
import com.speaksafe.ml.ModelEvaluator;
import com.speaksafe.ml.NaiveBayesClassifier;
import com.speaksafe.ml.SvmClassifier;
import com.speaksafe.model.Action;
import com.speaksafe.model.AnalysisResult;
import com.speaksafe.model.Category;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.stereotype.Service;

import java.util.List;
import java.util.Map;

/**
 * MODULE: Categorization + Scoring.
```

```
 * Trains the chosen model when the app starts, then for every message:
 *   1. gets category probabilities from the model
 *   2. computes a toxicity score from 0 to 100
 *   3. picks the category label and the action (NONE / WARN / BLUR)
 */
@Service
public class ToxicityDetector {

    private final DetectionPipeline pipeline;
    private final List<DatasetLoader.Sample> dataset;
    private final int warnThreshold;
    private final int blurThreshold;

    public ToxicityDetector(@Value("${speaksafe.model:svm}") String modelName,
                            @Value("${speaksafe.warn-threshold:50}") int warnThreshold,
                            @Value("${speaksafe.blur-threshold:70}") int blurThreshold)
{
        this.warnThreshold = warnThreshold;
        this.blurThreshold = blurThreshold;
        this.dataset = DatasetLoader.load(DatasetLoader.DEFAULT_DATASET);
        this.pipeline = new DetectionPipeline(
                modelName.equalsIgnoreCase("naive-bayes") ? new NaiveBayesClassifier() :
new SvmClassifier());
        pipeline.train(dataset);
        System.out.println("SpeakSafe: trained " + pipeline.modelName() + " on " +
dataset.size() + " messages");
    }

    public AnalysisResult analyze(String text) {
        Map<Category, Double> probabilities = pipeline.predict(text);

        // Toxicity score = how sure the model is that the message is NOT safe.
        int score = (int) Math.round((1.0 - probabilities.get(Category.SAFE)) * 100);

        // Below the warning threshold the message is SAFE.
        // Otherwise it gets the most likely toxic category (insult, threat or hate
speech).
        Category category = Category.SAFE;
        if (score >= warnThreshold) {
            double best = -1;
            for (Map.Entry<Category, Double> e : probabilities.entrySet()) {
                if (e.getKey() != Category.SAFE && e.getValue() > best) {
                    best = e.getValue();
                    category = e.getKey();
                }
            }
        }

        return new AnalysisResult(text, score, category, decideAction(category, score),
                probabilities, pipeline.modelName());
    }

    /** MODULE: Action. Decides what the browser extension should do. */
    private Action decideAction(Category category, int score) {
        if (category == Category.SAFE) return Action.NONE;
        if (score >= blurThreshold) return Action.BLUR;
        return Action.WARN;
    }
```

```
    /** Evaluates both models on the dataset (for the results page). */
    public List<ModelEvaluator.EvaluationResult> evaluateModels() {
        return List.of(
                ModelEvaluator.evaluate(NaiveBayesClassifier::new, dataset),
                ModelEvaluator.evaluate(SvmClassifier::new, dataset));
    }
}
```

MODULE 7 – DATABASE MODULE

schema.sql

```
CREATE TABLE IF NOT EXISTS flagged_messages (
    id          INTEGER PRIMARY KEY AUTOINCREMENT,
    text        TEXT    NOT NULL,
    score       INTEGER NOT NULL,
    category    TEXT    NOT NULL,
    source_url  TEXT,
    reported_by TEXT    NOT NULL,   -- AUTO (flagged by the system) or USER (reported by
a person)
    created_at  TEXT    NOT NULL
);
CREATE UNIQUE INDEX IF NOT EXISTS idx_unique_flag
    ON flagged_messages (text, source_url, reported_by);
```

ReportRepository.java

```
package com.speaksafe.service;

import com.speaksafe.model.Category;
import com.speaksafe.model.Report;
import org.springframework.jdbc.core.JdbcTemplate;
import org.springframework.stereotype.Repository;

import java.time.LocalDateTime;
import java.time.format.DateTimeFormatter;
import java.util.LinkedHashMap;
import java.util.List;
import java.util.Map;

/**
 * MODULE: Database.
 * Saves flagged / reported messages in SQLite so moderators can review them.
 */
@Repository
public class ReportRepository {

    private static final DateTimeFormatter TIME = DateTimeFormatter.ofPattern("yyyy-MM-
dd HH:mm:ss");

    private final JdbcTemplate jdbc;

    public ReportRepository(JdbcTemplate jdbc) {
        this.jdbc = jdbc;
    }
```

```
    /** Saves a message. The same message from the same page is stored only once. */
    public void save(String text, int score, Category category, String sourceUrl, String
reportedBy) {
        jdbc.update("INSERT OR IGNORE INTO flagged_messages "
                        + "(text, score, category, source_url, reported_by, created_at)
VALUES (?, ?, ?, ?, ?, ?)",
                text, score, category.name(), sourceUrl == null ? "" : sourceUrl,
reportedBy,
                LocalDateTime.now().format(TIME));
    }

    public List<Report> findAll() {
        return jdbc.query("SELECT * FROM flagged_messages ORDER BY id DESC LIMIT 500",
                (rs, row) -> new Report(
                        rs.getLong("id"),
                        rs.getString("text"),
                        rs.getInt("score"),
                        Category.valueOf(rs.getString("category")),
                        rs.getString("source_url"),
                        rs.getString("reported_by"),
                        rs.getString("created_at")));
    }

    public void delete(long id) {
        jdbc.update("DELETE FROM flagged_messages WHERE id = ?", id);
    }

    /** Number of stored messages per category, e.g. {INSULT=4, THREAT=1, ...}. */
    public Map<String, Integer> countByCategory() {
        Map<String, Integer> counts = new LinkedHashMap<>();
        for (Category c : Category.values()) counts.put(c.name(), 0);
        jdbc.query("SELECT category, COUNT(*) AS total FROM flagged_messages GROUP BY
category",
                rs -> { counts.put(rs.getString("category"), rs.getInt("total")); });
        return counts;
    }
}
```

MODULE 8 – INPUT MODULE (REST API)

DetectionController.java

```
package com.speaksafe.controller;

import com.speaksafe.ml.ModelEvaluator;
import com.speaksafe.model.Action;
import com.speaksafe.model.AnalysisResult;
import com.speaksafe.model.Report;
import com.speaksafe.service.ReportRepository;
import com.speaksafe.service.ToxicityDetector;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.CrossOrigin;
import org.springframework.web.bind.annotation.DeleteMapping;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
```

```
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;

import java.util.ArrayList;
import java.util.List;
import java.util.Map;

/**
 * MODULE: Input (REST API).
 * The browser extension and the web page send messages here.
 */
@RestController
@RequestMapping("/api")
@CrossOrigin
public class DetectionController {

    private static final int MAX_LENGTH = 5000;
    private static final int MAX_BATCH = 50;

    private final ToxicityDetector detector;
    private final ReportRepository reports;

    public DetectionController(ToxicityDetector detector, ReportRepository reports) {
        this.detector = detector;
        this.reports = reports;
    }

    /** Request bodies. autoFlag = save the message automatically if it gets blurred. */
    public record AnalyzeRequest(String text, String sourceUrl, boolean autoFlag) { }
    public record BatchRequest(List<String> texts, String sourceUrl, boolean autoFlag) {
}
    public record ReportRequest(String text, String sourceUrl) { }

    @GetMapping("/health")
    public Map<String, String> health() {
        return Map.of("status", "UP");
    }

    /** Analyze one message. */
    @PostMapping("/analyze")
    public ResponseEntity<AnalysisResult> analyze(@RequestBody AnalyzeRequest request) {
        if (request.text() == null || request.text().isBlank()) {
            return ResponseEntity.badRequest().build();
        }
        return ResponseEntity.ok(analyzeAndFlag(request.text(), request.sourceUrl(),
request.autoFlag()));
    }

    /** Analyze many messages at once (used when the extension scans a page). */
    @PostMapping("/analyze/batch")
    public ResponseEntity<List<AnalysisResult>> analyzeBatch(@RequestBody BatchRequest
request) {
        if (request.texts() == null || request.texts().size() > MAX_BATCH) {
            return ResponseEntity.badRequest().build();
        }
        List<AnalysisResult> results = new ArrayList<>();
```

```
        for (String text : request.texts()) {
            results.add(analyzeAndFlag(text == null ? "" : text, request.sourceUrl(),
request.autoFlag()));
        }
        return ResponseEntity.ok(results);
    }

    /** A person reports a message to the moderators. */
    @PostMapping("/report")
    public ResponseEntity<AnalysisResult> report(@RequestBody ReportRequest request) {
        if (request.text() == null || request.text().isBlank()) {
            return ResponseEntity.badRequest().build();
        }
        String text = limit(request.text());
        AnalysisResult result = detector.analyze(text);
        reports.save(text, result.score(), result.category(), request.sourceUrl(),
"USER");
        return ResponseEntity.ok(result);
    }

    @GetMapping("/reports")
    public List<Report> listReports() {
        return reports.findAll();
    }

    @DeleteMapping("/reports/{id}")
    public void deleteReport(@PathVariable long id) {
        reports.delete(id);
    }

    @GetMapping("/stats")
    public Map<String, Integer> stats() {
        return reports.countByCategory();
    }

    /** Accuracy, precision, recall and F1 for Naive Bayes vs SVM. */
    @GetMapping("/evaluate")
    public List<ModelEvaluator.EvaluationResult> evaluate() {
        return detector.evaluateModels();
    }

    private AnalysisResult analyzeAndFlag(String text, String sourceUrl, boolean
autoFlag) {
        text = limit(text);
        AnalysisResult result = detector.analyze(text);
        if (autoFlag && result.action() == Action.BLUR) {
            reports.save(text, result.score(), result.category(), sourceUrl, "AUTO");
        }
        return result;
    }

    private String limit(String text) {
        return text.length() > MAX_LENGTH ? text.substring(0, MAX_LENGTH) : text;
    }
}
```

MODULE 9 – MAIN APPLICATION

SpeakSafeApplication.java

```
package com.speaksafe;

import com.speaksafe.ml.ModelEvaluator;
import com.speaksafe.ml.ModelExporter;
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

/**
 * Entry point of SpeakSafe.
 *
 *   java -jar speaksafe.jar             -> starts the REST API on http://localhost:8080
 *   java -jar speaksafe.jar --evaluate  -> trains both models and prints accuracy,
precision, recall, F1
 *   java -jar speaksafe.jar --export-model model.json -> saves the trained model for
the browser extension
 */
@SpringBootApplication
public class SpeakSafeApplication {

    public static void main(String[] args) throws Exception {
        for (int i = 0; i < args.length; i++) {
            if (args[i].equals("--evaluate")) {
                ModelEvaluator.printComparison();
                return;
            }
            if (args[i].equals("--export-model") && i + 1 < args.length) {
                ModelExporter.export(args[i + 1], 50, 70);
                return;
            }
        }
        SpringApplication.run(SpeakSafeApplication.class, args);
    }
}
```

application.properties

```
# ---------- Server ----------
server.port=8080

# ---------- Database (SQLite file created next to where you run the app) ----------
spring.datasource.url=jdbc:sqlite:speaksafe.db
spring.datasource.driver-class-name=org.sqlite.JDBC
spring.datasource.hikari.maximum-pool-size=1
spring.sql.init.mode=always

# ---------- SpeakSafe settings ----------
# Which model scores messages: naive-bayes or svm
speaksafe.model=svm
# Score (0-100) at which the user is warned
speaksafe.warn-threshold=50
# Score (0-100) at which a message is blurred and flagged
speaksafe.blur-threshold=70
```

MODULE 10 – BROWSER EXTENSION

manifest.json

```
{
  "manifest_version": 3,
  "name": "SpeakSafe - Cyberbullying & Hate Speech Detector",
  "version": "1.1.0",
  "description": "Warns you before you send hurtful messages and blurs cyberbullying and
hate speech on web pages. Works offline with a built-in model.",
  "permissions": ["storage"],
  "host_permissions": ["http://localhost:8080/*"],
  "background": {
    "service_worker": "background.js"
  },
  "content_scripts": [
    {
      "matches": ["<all_urls>"],
      "js": ["content.js"],
      "css": ["content.css"],
      "run_at": "document_idle"
    }
  ],
  "action": {
    "default_title": "SpeakSafe",
    "default_popup": "popup.html",
    "default_icon": { "16": "icons/icon16.png", "48": "icons/icon48.png", "128":
"icons/icon128.png" }
  },
  "icons": { "16": "icons/icon16.png", "48": "icons/icon48.png", "128":
"icons/icon128.png" }
}
```

background.js

```
// SpeakSafe background service worker.
// Content scripts cannot call the Java backend directly (web pages block it),
// so they send messages here.
//
// Two modes:
//   1. Java server running on localhost:8080 -> it scores messages and stores reports
in SQLite
//   2. No server -> the built-in model (same trained SVM, exported to model.json)
scores messages
//      inside the browser and reports are kept in the extension's own storage

importScripts('speaksafe-model.js');

const SERVER = 'http://localhost:8080';
const RETRY_SERVER_AFTER_MS = 30000;   // when the server is down, check again after 30
s
const MAX_LOCAL_REPORTS = 500;

const DEFAULT_SETTINGS = {
  enabled: true,      // master on/off switch
  warnTyping: true,   // warn the user while typing
```

```
  scanPage: true,     // blur toxic messages on the page
  autoFlag: true      // save blurred messages for moderators
};

chrome.runtime.onInstalled.addListener(async () => {
  const saved = await chrome.storage.sync.get(DEFAULT_SETTINGS);
  await chrome.storage.sync.set(saved);
  await chrome.storage.local.set({ checked: 0, blurred: 0, warned: 0, reported: 0 });
});

// ---------------- built-in model ----------------
let modelPromise = null;
function builtInModel() {
  if (!modelPromise) {
    modelPromise = fetch(chrome.runtime.getURL('model.json'))
      .then(response => response.json())
      .then(json => new SpeakSafeModel(json));
  }
  return modelPromise;
}

// ---------------- Java server ----------------
let serverDownSince = 0;

async function callServer(path, body) {
  if (serverDownSince && Date.now() - serverDownSince < RETRY_SERVER_AFTER_MS) {
    throw new Error('Server offline');
  }
  try {
    const response = await fetch(SERVER + path, {
      method: body ? 'POST' : 'GET',
      headers: { 'Content-Type': 'application/json' },
      body: body ? JSON.stringify(body) : undefined,
      signal: AbortSignal.timeout(3000)
    });
    if (!response.ok) throw new Error('Server error ' + response.status);
    serverDownSince = 0;
    return await response.json();
  } catch (error) {
    serverDownSince = Date.now();
    throw error;
  }
}

// ---------------- reports kept in the extension (when there is no server)
----------------
async function saveLocalReport(result, url, reportedBy) {
  const { localReports = [] } = await chrome.storage.local.get('localReports');
  const duplicate = localReports.some(r => r.text === result.text && r.sourceUrl === url
&& r.reportedBy === reportedBy);
  if (duplicate) return;
  localReports.unshift({
    id: Date.now() + Math.random(),
    text: result.text,
    score: result.score,
    category: result.category,
    sourceUrl: url || '',
    reportedBy: reportedBy,
```

```
    createdAt: new Date().toLocaleString()
  });
  await chrome.storage.local.set({ localReports: localReports.slice(0,
MAX_LOCAL_REPORTS) });
}

async function addToCounter(name, amount) {
  if (!amount) return;
  const counters = await chrome.storage.local.get({ [name]: 0 });
  await chrome.storage.local.set({ [name]: counters[name] + amount });
}

// ---------------- message handling ----------------
async function analyzeMany(texts, url, autoFlag) {
  try {
    return await callServer('/api/analyze/batch', { texts, sourceUrl: url, autoFlag });
  } catch (e) {
    const model = await builtInModel();
    const results = texts.map(t => model.analyze(t));
    if (autoFlag) {
      for (const r of results) if (r.action === 'BLUR') await saveLocalReport(r, url,
'AUTO');
    }
    return results;
  }
}

async function handle(message, sender) {
  switch (message.type) {
    case 'health':
      try {
        await callServer('/api/health');
        return { mode: 'server' };
      } catch (e) {
        await builtInModel();
        return { mode: 'built-in' };
      }

    case 'analyze': {
      let result;
      try {
        result = await callServer('/api/analyze', { text: message.text, sourceUrl:
message.url, autoFlag: false });
      } catch (e) {
        result = (await builtInModel()).analyze(message.text);
      }
      await addToCounter('checked', 1);
      if (result.action !== 'NONE') await addToCounter('warned', 1);
      return result;
    }

    case 'analyzeBatch': {
      const results = await analyzeMany(message.texts, message.url, message.autoFlag);
      const blurred = results.filter(r => r.action === 'BLUR').length;
      await addToCounter('checked', results.length);
      await addToCounter('blurred', blurred);
      // Show how many messages were hidden on this page as a red number on the icon.
      if (sender.tab && blurred > 0) {
```

```
        chrome.action.setBadgeBackgroundColor({ tabId: sender.tab.id, color:
'#dc2626' });
        chrome.action.setBadgeText({ tabId: sender.tab.id, text:
String(message.pageBlurredTotal + blurred) });
      }
      return results;
    }

    case 'report': {
      let result;
      try {
        result = await callServer('/api/report', { text: message.text, sourceUrl:
message.url });
      } catch (e) {
        result = (await builtInModel()).analyze(message.text);
        await saveLocalReport(result, message.url, 'USER');
      }
      await addToCounter('reported', 1);
      return result;
    }
  }
  throw new Error('Unknown message type: ' + message.type);
}

chrome.runtime.onMessage.addListener((message, sender, sendResponse) => {
  handle(message, sender)
    .then(data => sendResponse({ ok: true, data }))
    .catch(error => sendResponse({ ok: false, error: error.message }));
  return true;   // tells Chrome we will answer asynchronously
});
```

content.js

```
// SpeakSafe content script: runs inside every web page.
// 1. Warns the user while they type a hurtful message.
// 2. Scans the messages on the page and blurs toxic ones.

(() => {
  const BATCH_SIZE = 25;
  const MIN_LENGTH = 3;
  const MAX_LENGTH = 600;
  const LABELS = { INSULT: 'Insult', THREAT: 'Threat', HATE_SPEECH: 'Hate speech', SAFE:
'Safe' };

  let settings = { enabled: true, warnTyping: true, scanPage: true, autoFlag: true };
  let blurredOnPage = 0;
  const checkedText = new WeakMap();   // element -> text we already analyzed

  // Don't scan SpeakSafe's own dashboard, otherwise it would blur the reports.
  if (location.host === 'localhost:8080') return;

  // Sends a request to background.js and returns the backend's answer (or null if the
backend is offline).
  function ask(message) {
    return new Promise(resolve => {
      try {
        chrome.runtime.sendMessage(message, response => {
          resolve(response && response.ok ? response.data : null);
```

```
        });
      } catch (e) {
        resolve(null);   // extension was reloaded; ignore
      }
    });
  }

  // ---------------------------------------------------------------
  // PART 1: warn while typing
  // ---------------------------------------------------------------
  let bubble = null;
  let typingTimer = null;

  function isTextBox(el) {
    if (!el || !el.tagName) return false;
    if (el.tagName === 'TEXTAREA') return true;
    if (el.tagName === 'INPUT') return ['text', 'search',
''].includes((el.getAttribute('type') || '').toLowerCase());
    return el.isContentEditable;
  }

  function textOf(el) {
    return (el.value !== undefined ? el.value : el.innerText || '').trim();
  }

  function hideBubble() {
    if (bubble) bubble.remove();
    bubble = null;
  }

  function showBubble(target, result) {
    hideBubble();
    bubble = document.createElement('div');
    bubble.className = 'speaksafe-bubble ' + (result.action === 'BLUR' ? 'block' :
'warn');
    bubble.textContent = (result.action === 'BLUR' ? '⛔ ' : '⚠️ ')
      + 'SpeakSafe: this message looks like ' + LABELS[result.category].toLowerCase()
      + ' (toxicity ' + result.score + '/100). Please think before you send it.';
    document.body.appendChild(bubble);
    const box = target.getBoundingClientRect();
    bubble.style.left = (window.scrollX + box.left) + 'px';
    // Show the bubble below the text box, or above it if there is no room below (e.g.
chat apps).
    const fitsBelow = box.bottom + 6 + bubble.offsetHeight <= window.innerHeight;
    const top = fitsBelow ? box.bottom + 6 : box.top - 6 - bubble.offsetHeight;
    bubble.style.top = (window.scrollY + top) + 'px';
  }

  document.addEventListener('input', event => {
    const target = event.target.closest ? event.target.closest('textarea, input,
[contenteditable]') : null;
    if (!settings.enabled || !settings.warnTyping || !isTextBox(target)) return;
    clearTimeout(typingTimer);
    typingTimer = setTimeout(async () => {   // wait until the user pauses typing
      const text = textOf(target);
      if (text.length < MIN_LENGTH) { hideBubble(); return; }
      const result = await ask({ type: 'analyze', text: text.slice(0, 2000), url:
location.href });
```

```
      if (result && result.action !== 'NONE' && textOf(target) === text)
showBubble(target, result);
      else hideBubble();
    }, 700);
  }, true);

  document.addEventListener('focusout', () => setTimeout(hideBubble, 200), true);

  // ------------------------------------------------------------
  // PART 2: scan the page and blur toxic messages
  // ------------------------------------------------------------
  const SKIP = 'script, style, noscript, textarea, input, select, code, pre,
[contenteditable], .speaksafe-label, .speaksafe-bubble';

  // Finds the smallest "block" element (paragraph, list item, comment div, ...) around
a piece of text.
  function blockOf(node) {
    let el = node.parentElement;
    while (el && el !== document.body &&
getComputedStyle(el).display.startsWith('inline')) {
      el = el.parentElement;
    }
    return el;
  }

  function findMessages() {
    const blocks = new Set();
    const walker = document.createTreeWalker(document.body, NodeFilter.SHOW_TEXT);
    let node;
    while ((node = walker.nextNode()) && blocks.size < 300) {
      if (node.nodeValue.trim().length < MIN_LENGTH || node.parentElement.closest(SKIP))
continue;
      const block = blockOf(node);
      if (!block || block === document.body || block.closest(SKIP + ', .speaksafe-
blurred')) continue;
      blocks.add(block);
    }
    const messages = [];
    for (const block of blocks) {
      // Use only the innermost blocks, so a comment and its container are not both
blurred.
      if ([...blocks].some(other => other !== block && block.contains(other))) continue;
      if (block.querySelector('.speaksafe-blurred, .speaksafe-label')) continue;
      const text = (block.innerText || '').trim().replace(/\s+/g, ' ');
      if (text.length < MIN_LENGTH || text.length > MAX_LENGTH) continue;
      if (checkedText.get(block) === text) continue;   // already checked, nothing
changed
      checkedText.set(block, text);
      messages.push({ block, text });
    }
    return messages;
  }

  function blur(block, result) {
    if (block.classList.contains('speaksafe-blurred')) return;
    block.classList.add('speaksafe-blurred');

    const label = document.createElement('span');
```

```
    label.className = 'speaksafe-label';
    label.textContent = '🛡️ Hidden by SpeakSafe: ' + LABELS[result.category] + ' (' +
result.score + '/100)';

    const show = document.createElement('button');
    show.textContent = 'Show';
    show.onclick = () => {
      const hidden = block.classList.toggle('speaksafe-blurred');
      show.textContent = hidden ? 'Show' : 'Hide';
    };

    const report = document.createElement('button');
    report.textContent = 'Report';
    report.onclick = async () => {
      report.textContent = 'Reporting…';
      const ok = await ask({ type: 'report', text: result.text, url: location.href });
      report.textContent = ok ? 'Reported ✓' : 'Server offline';
      report.disabled = true;
    };

    label.append(show, report);
    block.parentNode.insertBefore(label, block);
  }

  function warn(block, result) {
    block.classList.add('speaksafe-warned');
    block.title = 'SpeakSafe: possible ' + LABELS[result.category].toLowerCase() + ' ('
+ result.score + '/100)';
  }

  let scanning = false;
  async function scanPage() {
    if (!settings.enabled || !settings.scanPage || scanning) return;
    scanning = true;
    try {
      const messages = findMessages();
      for (let i = 0; i < messages.length; i += BATCH_SIZE) {
        const batch = messages.slice(i, i + BATCH_SIZE);
        const results = await ask({
          type: 'analyzeBatch',
          texts: batch.map(m => m.text),
          url: location.href,
          autoFlag: settings.autoFlag,
          pageBlurredTotal: blurredOnPage
        });
        if (!results) return;   // backend offline
        results.forEach((result, j) => {
          if (result.action === 'BLUR') { blur(batch[j].block, result); blurredOnPage++;
}
          else if (result.action === 'WARN') warn(batch[j].block, result);
        });
      }
    } finally {
      scanning = false;
    }
  }
```

```
  // Re-scan when the page changes (new chat messages, new comments, infinite
scroll...).
  let scanTimer = null;
  const observer = new MutationObserver(mutations => {
    const ours = mutations.every(m => [...m.addedNodes].every(n =>
      n.classList && (n.classList.contains('speaksafe-label') ||
n.classList.contains('speaksafe-bubble'))));
    if (ours) return;
    clearTimeout(scanTimer);
    scanTimer = setTimeout(scanPage, 1000);
  });

  // ------------------------------------------------------------
  // Settings (changed from the popup)
  // ------------------------------------------------------------
  chrome.storage.sync.get(settings, saved => {
    settings = saved;
    scanPage();
    observer.observe(document.body, { childList: true, subtree: true, characterData:
true });
  });

  chrome.storage.onChanged.addListener((changes, area) => {
    if (area !== 'sync') return;
    for (const key in changes) settings[key] = changes[key].newValue;
    if (!settings.enabled || !settings.warnTyping) hideBubble();
    if (settings.enabled && settings.scanPage) scanPage();
  });
})();
```

popup.js

```
// SpeakSafe popup: settings switches, counters, server status and a quick test box.

const SETTINGS = ['enabled', 'warnTyping', 'scanPage', 'autoFlag'];
const COUNTERS = ['checked', 'blurred', 'warned', 'reported'];

function ask(message) {
  return new Promise(resolve => chrome.runtime.sendMessage(message, response =>
    resolve(response && response.ok ? response.data : null)));
}

async function loadSettings() {
  const saved = await chrome.storage.sync.get({ enabled: true, warnTyping: true,
scanPage: true, autoFlag: true });
  for (const key of SETTINGS) {
    const box = document.getElementById(key);
    box.checked = saved[key];
    box.addEventListener('change', () => chrome.storage.sync.set({ [key]:
box.checked }));
  }
}

async function loadCounters() {
  const counts = await chrome.storage.local.get({ checked: 0, blurred: 0, warned: 0,
reported: 0 });
  for (const key of COUNTERS) document.getElementById(key).textContent = counts[key];
}
```

```
async function checkServer() {
  const status = document.getElementById('status');
  const link = document.getElementById('dashboardLink');
  const health = await ask({ type: 'health' });
  if (health && health.mode === 'server') {
    status.textContent = 'Connected to Java server';
    status.className = 'status online';
    link.href = 'http://localhost:8080/dashboard.html';
    link.textContent = 'Open moderator dashboard →';
  } else {
    // No server needed: the built-in model runs inside the browser.
    status.textContent = 'Built-in model active (no server needed)';
    status.className = 'status builtin';
    link.href = chrome.runtime.getURL('reports.html');
    link.textContent = 'Open flagged messages →';
  }
}

async function testMessage() {
  const text = document.getElementById('testText').value.trim();
  const output = document.getElementById('testResult');
  if (!text) return;
  const result = await ask({ type: 'analyze', text: text, url: 'popup' });
  if (!result) {
    output.textContent = 'Server offline';
    output.className = 'result';
    return;
  }
  output.textContent = result.category.replace('_', ' ') + ' – toxicity ' + result.score
+ '/100';
  output.className = 'result ' + result.category;
  loadCounters();
}

document.getElementById('testBtn').addEventListener('click', testMessage);
loadSettings();
loadCounters();
checkServer();
```

MODULE 11 – BUILT-IN BROWSER MODEL

speaksafe-model.js

```
// SpeakSafe built-in model (JavaScript version of the Java detection engine).
//
// The Java backend exports its trained Linear SVM to model.json
// (java -jar speaksafe.jar --export-model model.json). This file repeats the
// same steps in the browser, so the extension and the web demo work without the server:
//   clean text -> words -> word stems -> TF-IDF -> SVM -> score 0-100 -> category ->
action
// Every step mirrors a Java class (named in the comments) so both give the same scores.

(function (root) {
  'use strict';
```

```
  // ---------------------------------------------------------------
  // Porter stemmer: a line-by-line port of opennlp.tools.stemmer.PorterStemmer
  // ---------------------------------------------------------------
  function porterStem(word) {
    const b = word.split('');
    let j = 0, k = b.length - 1, dirty = false;
    const k0 = 0;

    const cons = i => {
      switch (b[i]) {
        case 'a': case 'e': case 'i': case 'o': case 'u': return false;
        case 'y': return i === k0 || !cons(i - 1);
        default: return true;
      }
    };
    const m = () => {
      let n = 0, i = k0;
      while (true) { if (i > j) return n; if (!cons(i)) break; i++; }
      i++;
      while (true) {
        while (true) { if (i > j) return n; if (cons(i)) break; i++; }
        i++; n++;
        while (true) { if (i > j) return n; if (!cons(i)) break; i++; }
        i++;
      }
    };
    const vowelInStem = () => { for (let i = k0; i <= j; i++) if (!cons(i)) return true;
return false; };
    const doublec = jj => jj >= k0 + 1 && b[jj] === b[jj - 1] && cons(jj);
    const cvc = i => {
      if (i < k0 + 2 || !cons(i) || cons(i - 1) || !cons(i - 2)) return false;
      const ch = b[i];
      return !(ch === 'w' || ch === 'x' || ch === 'y');
    };
    const ends = s => {
      const l = s.length, o = k - l + 1;
      if (o < k0) return false;
      for (let i = 0; i < l; i++) if (b[o + i] !== s[i]) return false;
      j = k - l;
      return true;
    };
    const setto = s => {
      for (let i = 0; i < s.length; i++) b[j + 1 + i] = s[i];
      k = j + s.length;
      dirty = true;
    };
    const r = s => { if (m() > 0) setto(s); };

    function step1() {
      if (b[k] === 's') {
        if (ends('sses')) k -= 2;
        else if (ends('ies')) setto('i');
        else if (b[k - 1] !== 's') k--;
      }
      if (ends('eed')) {
        if (m() > 0) k--;
      } else if ((ends('ed') || ends('ing')) && vowelInStem()) {
        k = j;
```

```
        if (ends('at')) setto('ate');
        else if (ends('bl')) setto('ble');
        else if (ends('iz')) setto('ize');
        else if (doublec(k)) {
          const ch = b[k--];
          if (ch === 'l' || ch === 's' || ch === 'z') k++;
        } else if (m() === 1 && cvc(k)) setto('e');
      }
    }
    function step2() {
      if (ends('y') && vowelInStem()) { b[k] = 'i'; dirty = true; }
    }
    function tryRules(rules) {
      for (const [suffix, replacement] of rules) {
        if (ends(suffix)) { r(replacement); return; }
      }
    }
    function step3() {
      if (k === k0) return;
      const rules = {
        a: [['ational', 'ate'], ['tional', 'tion']],
        c: [['enci', 'ence'], ['anci', 'ance']],
        e: [['izer', 'ize']],
        l: [['bli', 'ble'], ['alli', 'al'], ['entli', 'ent'], ['eli', 'e'], ['ousli',
'ous']],
        o: [['ization', 'ize'], ['ation', 'ate'], ['ator', 'ate']],
        s: [['alism', 'al'], ['iveness', 'ive'], ['fulness', 'ful'], ['ousness',
'ous']],
        t: [['aliti', 'al'], ['iviti', 'ive'], ['biliti', 'ble']],
        g: [['logi', 'log']]
      }[b[k - 1]];
      if (rules) tryRules(rules);
    }
    function step4() {
      const rules = {
        e: [['icate', 'ic'], ['ative', ''], ['alize', 'al']],
        i: [['iciti', 'ic']],
        l: [['ical', 'ic'], ['ful', '']],
        s: [['ness', '']]
      }[b[k]];
      if (rules) tryRules(rules);
    }
    function step5() {
      if (k === k0) return;
      let found = false;
      switch (b[k - 1]) {
        case 'a': found = ends('al'); break;
        case 'c': found = ends('ance') || ends('ence'); break;
        case 'e': found = ends('er'); break;
        case 'i': found = ends('ic'); break;
        case 'l': found = ends('able') || ends('ible'); break;
        case 'n': found = ends('ant') || ends('ement') || ends('ment') || ends('ent');
break;
        case 'o': found = (ends('ion') && j >= 0 && (b[j] === 's' || b[j] === 't')) ||
ends('ou'); break;
        case 's': found = ends('ism'); break;
        case 't': found = ends('ate') || ends('iti'); break;
        case 'u': found = ends('ous'); break;
```

```
        case 'v': found = ends('ive'); break;
        case 'z': found = ends('ize'); break;
      }
      if (found && m() > 1) k = j;
    }
    function step6() {
      j = k;
      if (b[k] === 'e') {
        const a = m();
        if (a > 1 || (a === 1 && !cvc(k - 1))) k--;
      }
      if (b[k] === 'l' && doublec(k) && m() > 1) k--;
    }

    const length = b.length;
    if (k > k0 + 1) { step1(); step2(); step3(); step4(); step5(); step6(); }
    if (length !== k + 1) dirty = true;
    return dirty ? b.slice(0, k + 1).join('') : word;
  }

  // ------------------------------------------------------------
  // Preprocessing (same as TextPreprocessor.java)
  // ------------------------------------------------------------
  const JAVA_SPACE = '[ \\t\\n\\x0B\\f\\r]';   // Java's \s (ASCII whitespace only)

  function expandContractions(text) {
    return text.replace(/\u2019/g, "'")
      .replace(/\bcan't\b/g, 'can not')
      .replace(/\bwon't\b/g, 'will not')
      .replace(/n't\b/g, ' not')
      .replace(/'re\b/g, ' are')
      .replace(/'ll\b/g, ' will')
      .replace(/'m\b/g, ' am')
      .replace(/'ve\b/g, ' have')
      .replace(/'d\b/g, ' would')
      .replace(/'s\b/g, '');
  }

  function normalizeLeetspeak(text) {
    const words = text.split(new RegExp(JAVA_SPACE + '+'));
    while (words.length && words[words.length - 1] === '') words.pop();   // Java's
split() drops trailing empties
    let result = '';
    for (let word of words) {
      if (/[a-z]/.test(word)) {
        word = word.replace(/0/g, 'o').replace(/1/g, 'i').replace(/3/g, 'e')
                   .replace(/4/g, 'a').replace(/5/g, 's').replace(/7/g, 't')
                   .replace(/@/g, 'a').replace(/\$/g, 's')
                   .replace(/\*/g, '');                                    // f*ck ->
fck
      }
      result += word + ' ';
    }
    return result;
  }

  function preprocess(text, stopwords, slang) {
    slang = slang || {};
```

```
    if (text == null) return [];
    let clean = String(text).toLowerCase();
    clean = clean.replace(/https?:\/\/[^ \t\n\x0B\f\r]+|www\.[^ \t\n\x0B\f\r]+/g, ' ');
// remove links
    clean = clean.replace(/@[a-zA-Z0-9_]+/g, ' ');
// remove @mentions
    clean =
expandContractions(clean);                                                  // you're ->
you are
    clean =
normalizeLeetspeak(clean);                                                  // 1d10t ->
idiot
    clean = clean.replace(/([^\n\r\u0085\u2028\u2029])\1{2,}/gu,
'$1');                 // stuuupid -> stupid
    clean = clean.replace(/[^a-z \t\n\x0B\f\r]/g, '
');                                 // remove symbols and digits
    const tokens = [];
    for (let token of clean.split(new RegExp(JAVA_SPACE + '+'))) {
      if (Object.prototype.hasOwnProperty.call(slang, token)) token = slang[token];   //
f*ck / fck -> fuck
      if (token.length > 1 && !stopwords.has(token)) tokens.push(porterStem(token));
    }
    return tokens;
  }

  // ---------------------------------------------------------------
  // The model: TF-IDF (TfIdfVectorizer.java) + Linear SVM (SvmClassifier.java)
  // + scoring, category and action (ToxicityDetector.java)
  // ---------------------------------------------------------------
  class SpeakSafeModel {
    constructor(json) {
      this.name = json.name;
      this.categories = json.categories;               // [SAFE, INSULT, THREAT,
HATE_SPEECH]
      this.stopwords = new Set(json.stopwords);
      this.slang = json.slang || {};
      this.vocabulary = new Map(json.terms.map((term, i) => [term, i]));
      this.idf = json.idf;
      this.weights = json.weights;
      this.bias = json.bias;
      this.warnThreshold = json.warnThreshold;
      this.blurThreshold = json.blurThreshold;
    }

    vectorize(words) {
      const terms = words.slice();
      for (let i = 0; i + 1 < words.length; i++) terms.push(words[i] + '_' + words[i +
1]);
      const vector = new Map();
      for (const term of terms) {
        const index = this.vocabulary.get(term);
        if (index !== undefined) vector.set(index, (vector.get(index) || 0) + 1);
      }
      let length = 0;
      for (const [index, tf] of vector) {
        const weight = tf * this.idf[index];
        vector.set(index, weight);
        length += weight * weight;
```

```
      }
      const norm = Math.sqrt(length);
      if (norm > 0) for (const [index, weight] of vector) vector.set(index, weight /
norm);
      return vector;
    }

    probabilities(vector) {
      const probabilities = {};
      if (vector.size === 0) {   // no known words -> safe
        this.categories.forEach(c => { probabilities[c] = c === 'SAFE' ? 1 : 0; });
        return probabilities;
      }
      const p = this.categories.map((c, ci) => {
        let score = this.bias[ci];
        for (const [index, value] of vector) score += this.weights[ci][index] * value;
        return 1 / (1 + Math.exp(-3 * score));
      });
      let sum = p.reduce((a, x) => a + x, 0);
      const safe = this.categories.indexOf('SAFE');
      if (sum < 1) { p[safe] += 1 - sum; sum = 1; }
      this.categories.forEach((c, ci) => { probabilities[c] = p[ci] / sum; });
      return probabilities;
    }

    analyze(text) {
      text = String(text == null ? '' : text).slice(0, 5000);
      const probabilities = this.probabilities(this.vectorize(preprocess(text,
this.stopwords, this.slang)));
      const score = Math.round((1 - probabilities.SAFE) * 100);
      let category = 'SAFE';
      if (score >= this.warnThreshold) {
        let best = -1;
        for (const c of this.categories) {
          if (c !== 'SAFE' && probabilities[c] > best) { best = probabilities[c];
category = c; }
        }
      }
      const action = category === 'SAFE' ? 'NONE' : score >= this.blurThreshold ? 'BLUR'
: 'WARN';
      return { text, score, category, action, probabilities, model: 'Linear SVM (built-
in)' };
    }
  }

  SpeakSafeModel.preprocess = preprocess;
  SpeakSafeModel.porterStem = porterStem;
  root.SpeakSafeModel = SpeakSafeModel;
})(typeof self !== 'undefined' ? self : globalThis);
```

APPENDIX 2
SAMPLE OUTPUT

Model evaluation (java -jar speaksafe.jar --evaluate)

```
================ SpeakSafe Model Evaluation ================
Dataset size: 1520 messages (80% train / 20% test)

Model           Accuracy  Precision   Recall  F1-score   ms/message
Naive Bayes        96.4%      96.7%    92.1%     94.2%        0.040
Linear SVM         95.7%      96.1%    91.2%     93.4%        0.039

--- Naive Bayes: per category ---
Category      Precision   Recall  F1-score  Support
SAFE              96.5%    99.0%     97.7%      196
INSULT            94.4%    98.1%     96.2%       52
THREAT            95.8%    82.1%     88.5%       28
HATE_SPEECH      100.0%    89.3%     94.3%       28
Confusion matrix (rows = actual, columns = predicted):
                    SAFE      INSULT      THREAT HATE_SPEECH
SAFE                 194           1           1           0
INSULT                 1          51           0           0
THREAT                 3           2          23           0
HATE_SPEECH            3           0           0          25

--- Linear SVM: per category ---
Category      Precision   Recall  F1-score  Support
SAFE              95.6%    99.0%     97.2%      196
INSULT            96.1%    94.2%     95.1%       52
THREAT           100.0%    82.1%     90.2%       28
HATE_SPEECH       92.6%    89.3%     90.9%       28
Confusion matrix (rows = actual, columns = predicted):
                    SAFE      INSULT      THREAT HATE_SPEECH
SAFE                 194           0           0           2
INSULT                 3          49           0           0
THREAT                 3           2          23           0
HATE_SPEECH            3           0           0          25
============================================================
```

REFERENCES

[1] A. Alabdulwahab, M. A. Haq and M. Alshehri, “Cyberbullying Detection using Machine Learning and Deep Learning,” International Journal of Advanced Computer Science and Applications, vol. 14, no. 10, 2023.

[2] “A Review on Deep-Learning-Based Cyberbullying Detection,” Future Internet, vol. 15, no. 5, p. 179, 2023.

[3] S. S. Nath, R. Karim and M. H. Miraz, “Deep Learning Based Cyberbullying Detection in Bangla Language,” Annals of Emerging Technologies in Computing (AETiC), vol. 8, no. 1, 2024.

[4] H. Khafajeh, “Cyberbullying Detection in Social Networks Using Deep Learning,” The International Arab Journal of Information Technology, vol. 21, no. 6, pp. 1054–1061, 2024.

[5] T. Davidson, D. Warmsley, M. Macy and I. Weber, “Automated Hate Speech Detection and the Problem of Offensive Language,” in Proc. International AAAI Conference on Web and Social Media (ICWSM), 2017.

[6] K. Dinakar, R. Reichart and H. Lieberman, “Modeling the Detection of Textual Cyberbullying,” in Proc. ICWSM Workshop on the Social Mobile Web, 2011.

[7] Z. Waseem and D. Hovy, “Hateful Symbols or Hateful People? Predictive Features for Hate Speech Detection on Twitter,” in Proc. NAACL Student Research Workshop, 2016.

[8] A. Schmidt and M. Wiegand, “A Survey on Hate Speech Detection using Natural Language Processing,” in Proc. Fifth International Workshop on Natural Language Processing for Social Media, 2017.

[9] E. Wulczyn, N. Thain and L. Dixon, “Ex Machina: Personal Attacks Seen at Scale,” in Proc. 26th International World Wide Web Conference (WWW), 2017.

[10] S. Salawu, Y. He and J. Lumsden, “Approaches to Automated Detection of Cyberbullying: A Survey,” IEEE Transactions on Affective Computing, vol. 11, no. 1, pp. 3–24, 2020.

[11] J. Devlin, M.-W. Chang, K. Lee and K. Toutanova, “BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding,” in Proc. NAACL-HLT, 2019.

[12] C. Cortes and V. Vapnik, “Support-Vector Networks,” Machine Learning, vol. 20, no. 3, pp. 273–297, 1995.

[13] S. Shalev-Shwartz, Y. Singer and N. Srebro, “Pegasos: Primal Estimated sub-GrAdient SOlver for SVM,” in Proc. 24th International Conference on Machine Learning (ICML), 2007.

[14] M. F. Porter, “An Algorithm for Suffix Stripping,” Program, vol. 14, no. 3, pp. 130–137, 1980.

[15] G. Salton and C. Buckley, “Term-Weighting Approaches in Automatic Text Retrieval,” Information Processing & Management, vol. 24, no. 5, pp. 513–523, 1988.

[16] C. D. Manning, P. Raghavan and H. Schütze, Introduction to Information Retrieval, Cambridge University Press, 2008.

[17] H. Schildt, Java: The Complete Reference, McGraw-Hill Education.

[18] Jigsaw / Conversation AI, “Toxic Comment Classification Challenge,” Kaggle, 2018.

[19] Apache Software Foundation, “Apache OpenNLP Developer Documentation,” https://opennlp.apache.org

[20] Spring, “Spring Boot Reference Documentation,” https://docs.spring.io/spring-boot/

[21] Google, “Chrome Extensions Documentation (Manifest V3),” https://developer.chrome.com/docs/extensions